

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

ae52321 Jebel-Quant/lasso 4 Nov 2026

cvxcla: An Operator-Based Critical Line Algorithm in Python

Thomas Schmelzer
Jebel Quant Research

Abstract

cvxcla is an open-source Python implementation of Markowitz's Critical Line Algorithm (CLA), which returns the *entire* mean–variance efficient frontier as a finite list of turning points, exact up to floating-point round-off. The turning points are Markowitz's; the package adds an interface to the covariance and a wider range of constraints. The algorithm reaches the covariance only through a small operator protocol, so a dense matrix, a diagonal-plus-low-rank risk model, a data matrix, or a user-written backend can be exchanged without touching the loop, and the same covariance yields the same turning points. Box bounds, linear equalities and inequalities, and a cap on gross exposure are supported, the last through a lift the package performs itself. The walk provably terminates under standard nondegeneracy assumptions, every turning point is checked for feasibility as it is found, an independent Karush–Kuhn–Tucker certificate passes on thirteen traces of up to 2560 assets, and inputs outside the supported domain stop with an error that names the cause. A structured backend pays where the free set is large and the solve dominates: on a synthetic factor market of 2560 assets where every asset is free at some point, the factor backend traces the frontier about 30 times faster than the dense one in under a hundredth of the memory. On a factor model of the S&P 500, where few assets are free at once, the gain is about 1.4 times, and the exact frontier costs an order of magnitude less than a grid of 100 portfolios from a general solver.

Keywords: Critical Line Algorithm, mean–variance optimization, efficient frontier, parametric quadratic programming, active-set method, factor covariance models, Python.

1. Introduction

The mean–variance portfolio selection problem (Markowitz 1952) asks for the allocation of capital across n assets that minimizes risk for a given level of expected return; sweeping the return level produces a one-parameter family of optimal portfolios whose risk–return

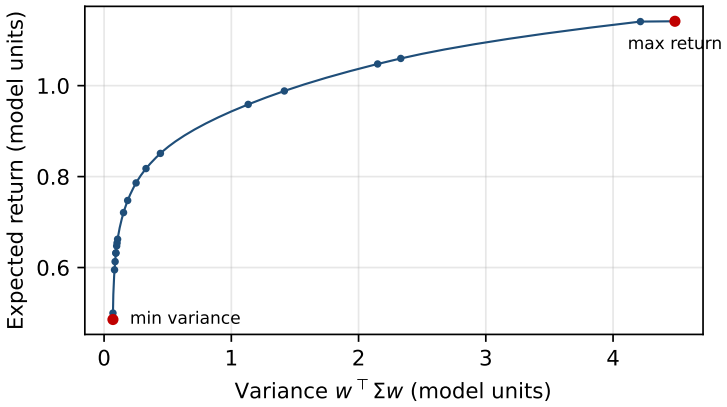


Figure 1: The efficient frontier of a 20-asset long-only problem, traced exactly by **cvxcla** as 20 turning points, plotted as return against variance, in which each segment between turning points is an arc of a parabola (`make_figures.py frontier`). The empirical figures use volatility, as is usual in practice.

image is the *efficient frontier*. The Critical Line Algorithm (CLA) of Markowitz (1956, 1959) represents this frontier *exactly and in full*: it returns a finite list of *turning points* between which the frontier is described in closed form, so no discretization or repeated re-optimization is required. The canonical description and a reference implementation are due to Markowitz and Todd (2000); Perold (1984), Stein, Branke, and Schmeck (2008), and Hirschberger, Qi, and Steuer (2010) develop large-scale and active-set variants, Niedermayer and Niedermayer (2010) a careful fast implementation, and Markowitz, Starer, Fram, and Gerber (2021) a recent review extending the method to mean–semivariance. In optimization terms the CLA solves a parametric quadratic program. Wolfe (1959) adapted the simplex method to quadratic programs, Best (1996) gave a parametric active-set algorithm that Best (2010) applies to portfolios, and **qpOASES** (Ferreau, Kirches, Potschka, Bock, and Diehl 2014) is a general parametric QP solver.

Classical CLA implementations take the covariance as a dense matrix. For a large universe it usually has more structure than that: multi-factor risk models, which generalize the single-index model (Sharpe 1963), write it as a diagonal matrix plus a low-rank term. The large-scale methods cited above exploit structure through incremental factorizations, sparsity, or pivoting tied to an explicit matrix. **cvxcla** computes the same turning points as any CLA; its contribution is a turning-point loop written so that it never needs the covariance as a matrix. Every step, including the start on a tied face, the gross-exposure lift and the singularity guard, uses only products with Σ or its blocks, solves against its free block and a condition estimate, so the structure becomes a property of an exchangeable backend. The operator protocol and the standard backends that implement these operations are provided by the companion package **cvx-linalg** (Schmelzer 2026c); **cvxcla** is the frontier algorithm on top of them (Section 5). The link between the CLA and the constrained LASSO is treated elsewhere (Schmelzer and Hastie 2026; Schmelzer 2026b,a); this paper documents the software.

Contributions.

Software	Output	Covariance	Constraints	Gross cap
CLA (R)	turning points	dense matrix	box bounds, one budget row	no
QP over a λ grid	one portfolio per grid point	as the solver accepts it	any convex	reformulated
qpOASES via **CasADi**	path followed, endpoint returned	matrix	general linear	reformulated
cvxcla	every turning point	any operator	box bounds, $Aw = b$, $Gw \leq h$	native lift

Table 1: What each route to the mean–variance frontier delivers, as used in Section 7: the QP over a λ grid is described in Section 7.5, **qpOASES** in Section 7.6, **cvxcla**'s covariance operators in Section 5 and its lift of the gross cap in Section 4.4. "Gross cap" is $\|w\|_1 \leq c$: "reformulated" means the user rewrites it with auxiliary variables before calling the solver, "native lift" that **cvxcla** accepts c and performs the rewriting itself. "Path followed, endpoint returned" means **qpOASES** passes the same turning points but does not expose them through **CasADi**. The termination guarantee (Section 4.5) is a property of **cvxcla**'s trace, and the KKT certificate (Section 7.1) a check of the replication script; neither is compared here.

1. **A CLA on an operator interface.** The turning-point loop, with its start on a tied face, its gross-exposure lift and its singularity guard, reaches the covariance only through the five members of the **cvx-linalg** protocol (Section 5), so dense, factor, data-matrix and user-written backends run the same loop and return the same turning points. With a diagonal-plus-low-rank backend the trace never forms an $n \times n$ matrix, not even for the lifted problem. Its gain grows with the free set: about 1.4× on a rank-20 factor model of the S&P 500, where at most 55 of 494 assets are free at once (Section 7.7), and about 30× at $n = 2560$, in under 1 MiB of working memory against 100 MiB, on a synthetic market whose free set grows to every asset (Section 7.2).
2. **General linear constraints.** Box bounds combine with any linear equalities $Aw = b$ and inequalities $Gw \leq h$ (Section 4.3) and a gross-exposure cap $\|w\|_1 \leq c$ (Section 4.4).
3. **A tested implementation.** A least-index tie-break makes the trace deterministic and provably finite under standard assumptions (Section 4.5); every turning point is checked for feasibility as it is found, and an independent KKT certificate in the replication script certifies thirteen traces, among them the largest timed one ($n = 2560$) and the worked example (Section 7.1); inputs outside the supported domain are declined with an error that names the cause (Section 6.3); and one script regenerates every figure and number.

Table 1 sets these capabilities against the software a user would otherwise reach for. The R package **CLA** (Shi and Maechler 2024), an open implementation of the CLA itself, covers box bounds and one budget row on a dense covariance, and **cvxcla** reproduces its turning points (Section 7.1). The `CLA` of **PyPortfolioOpt** (Martin 2021), with the same scope, is compared with **cvxcla** in a separate note (Schmelzer 2026e), on the turning points both return, their speed, and how each handles degenerate inputs; nothing in this paper depends on it.

We do *not* claim the fastest solve: on small problems **qpOASES** is faster (Section 7.6), and large dense problems admit faster incremental factorizations (Section 7.3), and on a factor

model whose free set grows to every asset an interior-point solver returns a grid of 100 approximate frontier portfolios at a similar cost, and in parallel faster (Section 7.5). The claim is an exact and complete frontier at no more than the cost of a grid; on the S&P 500, with few assets free at once, the grid costs an order of magnitude more.

The package (Schmelzer 2026d)[1] is built around the class `CLA` in `src/cvxcla/cla.py`, supported by `first.py` (the starting portfolio), `operators/` (the covariance backends, built on the operators of **cvx-linalg** (Schmelzer 2026c); Section 5), and `types.py` (the turning-point and frontier data structures). The notation follows the code.

Sections 2–4 describe the algorithm, Section 5 the operator interface, Section 6 the user interface and Section 7 the experiments; details and the termination proof are in the appendices.

2. Problem formulation

Let $w \in \mathbb{R}^n$ be the portfolio weights, $\mu \in \mathbb{R}^n$ the expected returns, $\Sigma \in \mathbb{R}^{n\times n}$ the (symmetric, positive semidefinite) covariance, $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$ the equality constraints, $G \in \mathbb{R}^{p\times n}$, $h \in \mathbb{R}^p$ the inequality constraints, and ℓ, u the bounds. The data μ, Σ, A, b, G, h must be finite. A bound may be infinite, an unbounded box; the first turning point is then found by the linear program of Section 3, which refuses a problem whose expected return is unbounded. The implementation traces the frontier through the *return-parametrized* quadratic program

$$\min_{w\in\mathbb{R}^n} \ \tfrac{1}{2}\, w^{\mathsf{T}}\Sigma\, w - \lambda\, \mu^{\mathsf{T}} w \quad \text{subject to} \quad Aw = b, \ \ Gw \le h, \ \ \ell \le w \le u, \tag{1}$$

where $\lambda \ge 0$ weights expected return against variance. It multiplies the return, not the risk, so it is the reciprocal of the usual risk-aversion parameter: large λ means a risk-tolerant investor. The canonical instance has the single budget row $\mathbf{1}^{\mathsf{T}} w = 1$; general A and G (group or sector caps, a $\ge$ row negated) are handled in Section 3 and Section 4.3. As λ decreases from $+\infty$ to 0, the optimiser moves from the maximum-return portfolio to the global minimum-variance portfolio, and its locus is the efficient frontier (Figure 1).

Optimality conditions. At fixed λ the program is convex, so the KKT conditions are necessary and sufficient. Partition the assets into a *free* set $\mathcal{F}$ (weights strictly inside their bounds) and a *blocked* set $\mathcal{B} = \mathcal{F}^c$ (weights at ℓ_i or u_i), and let $\mathcal{S} \subseteq \{1, \dots, p\}$ index the *active* inequality rows. For the free assets

$$\begin{aligned} \Sigma_{\mathcal{F}\mathcal{F}}\, w_{\mathcal{F}} + \Sigma_{\mathcal{F}\mathcal{B}}\, w_{\mathcal{B}} + A_{\mathcal{F}}^{\mathsf{T}}\nu + G_{\mathcal{S}\mathcal{F}}^{\mathsf{T}}\eta &= \lambda\, \mu_{\mathcal{F}}, \\ A_{\mathcal{F}}\, w_{\mathcal{F}} + A_{\mathcal{B}}\, w_{\mathcal{B}} = b, \qquad G_{\mathcal{S}\mathcal{F}}\, w_{\mathcal{F}} &+ G_{\mathcal{S}\mathcal{B}}\, w_{\mathcal{B}} = h_{\mathcal{S}}, \end{aligned} \tag{2}$$

with multipliers $\nu \in \mathbb{R}^m$ and $\eta \in \mathbb{R}^{|\mathcal{S}|}_{\ge 0}$. The blocked weights are known, so (2) is linear in $(w_{\mathcal{F}}, \nu, \eta)$, and an active inequality row enters exactly as an equality row: stacking $C = [\,A;\, G_{\mathcal{S}}\,]$ recovers the equality-only form.

The affine segment. While the partition is fixed, the system matrix is constant and the right-hand side affine in λ, so

$$w(\lambda) = \alpha + \lambda\, \beta, \tag{3}$$

[1] `github.com/cvxgrp/cvxcla`

with α, β constant on the segment ($\beta_{\mathcal{B}} = 0$, $\alpha_{\mathcal{B}} = w_{\mathcal{B}}$); the code calls them `r_alpha` and `r_beta`. This is the *critical line*. A *turning point* is a value of λ at which the partition must change.

3. The first turning point

At $\lambda = +\infty$ variance is irrelevant and (1) reduces to earning the most expected return the constraints allow. For a single all-ones budget row $\mathbf{1}^\mathsf{T} w = b$, `init_algo` (`first.py`) computes this greedily: start every weight at ℓ, sort the assets by descending μ, and raise each towards u_i until the budget is met (within a tolerance of 10^{-12}; without it, round-off can skip the asset that should end strictly inside its box). The last asset touched absorbs the residual and is the single *free* asset; all others are blocked. The same fill serves a budget above one ($b > 1$) and dollar-neutral books ($b = 0$, negative ℓ); "leverage" in the interface means the gross-exposure cap of Section 4.4. The last asset is marked free even when it lands exactly on a bound, so the reduced KKT system is non-singular from the outset.

For a general equality system (weighted rows, or $m > 1$ rows such as sector or factor neutrality) or any G, the maximum-return vertex is the linear program $\max_w \mu^\mathsf{T} w$ subject to the constraints, which **cvxcla** solves with the **HiGHS** simplex through `scipy.optimize.linprog`; the free set is read off as the assets strictly inside their bounds. Only this first step differs; the loop that follows is written for a general A throughout.

Tied maximum returns. The maximum-return set need not be a single vertex. When two or more assets share the largest expected return, say because they were given the same forecast, the linear program has a whole face of optima, and the greedy fill or the simplex returns one of its vertices. The frontier does not start at an arbitrary one. For λ just below $+\infty$ the optimiser of (1) is the point of that face where the quadratic term $\frac{1}{2} w^\mathsf{T} \Sigma w$ is smallest, and the path leaves from there. **cvxcla** starts at that point. It finds it with a second trace, run on the face itself (the constraints plus the row $\mu^\mathsf{T} w = \mu^\mathsf{T} w_0$, with w_0 the vertex the solver returned): that trace minimizes the variance over the face at its own $\lambda = 0$, and its own start is made unique by a generic linear term drawn from a fixed seed, so the result is deterministic. Where Σ fails the conditioning test of the singularity guard, it is first shifted by a multiple of $\mu\mu^\mathsf{T}$, which changes the objective on the face by a constant only and removes a singularity along directions that change the return (`CLA._face_minimiser` in `cla.py`). The free set is the assets strictly inside their bounds at the point found. The vertex the solver returns is in general not that point, and a walk started there does not lie on the path: on the tied problems of Appendix B it is declined with an error.

The vertex itself can be degenerate, with every weight on a bound or a duplicated tight row. Its partition is then read from a vertex of the linear program's optimal dual set, which keeps it dual feasible and starts the first segment at the vertex itself (Appendix A.3). Redundant equality rows, a repeated budget say, are dropped before the trace starts; a vertex is refused with a `DegenerateProblemError` (Section 6.3) only when its optimal dual set has no vertex.

4. The turning-point iteration

From the first turning point the algorithm repeatedly (i) solves the reduced KKT system for

the affine segment (3); (ii) finds the largest λ below the current one at which an *event* occurs; (iii) flips the status of the single coordinate responsible; and (iv) records the new turning point, stopping at $\lambda = 0$.

4.1. Solving the reduced KKT system by block elimination

Write $C_{\mathcal{F}}$ for the active constraint rows restricted to the free set ($A_{\mathcal{F}}$ when $\mathcal{S} = \varnothing$; Section 4.3) and c for their right-hand side. The reduced system is the saddle-point problem

$$\begin{bmatrix} \Sigma_{\mathcal{F}\mathcal{F}} & C_{\mathcal{F}}^{\mathsf{T}} \\ C_{\mathcal{F}} & 0 \end{bmatrix} \begin{bmatrix} w_{\mathcal{F}} \\ \nu \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix}, \tag{4}$$

solved for two right-hand sides at once, the "α system" $r_1^{\alpha} = -\Sigma_{\mathcal{F}\mathcal{B}} w_{\mathcal{B}}$, $r_2^{\alpha} = c - C_{\mathcal{B}} w_{\mathcal{B}}$, and the "$\beta$ system" $r_1^{\beta} = \mu_{\mathcal{F}}$, $r_2^{\beta} = 0$. Rather than factor the full saddle matrix, the code eliminates the free block. A single multi-column solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ gives $Y = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} C_{\mathcal{F}}^{\mathsf{T}}$ and $z^{\alpha,\beta} = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^{\alpha,\beta}$. The multipliers come from the small Schur complement $S = C_{\mathcal{F}} Y$. With $n_{\mathcal{F}}$ free assets and $m_c = m + |\mathcal{S}|$ active rows, $C_{\mathcal{F}} \in \mathbb{R}^{m_c \times n_{\mathcal{F}}}$, $Y \in \mathbb{R}^{n_{\mathcal{F}} \times m_c}$ and $S \in \mathbb{R}^{m_c \times m_c}$,

$$\nu^{\alpha} = S^{-1}(C_{\mathcal{F}} z^{\alpha} - r_2^{\alpha}), \qquad \nu^{\beta} = S^{-1} C_{\mathcal{F}} z^{\beta}, \tag{5}$$

and back-substitution gives

$$\alpha_{\mathcal{F}} = z^{\alpha} - Y\nu^{\alpha}, \qquad \beta_{\mathcal{F}} = z^{\beta} - Y\nu^{\beta}. \tag{6}$$

The dominant cost is the solve against $\Sigma_{\mathcal{F}\mathcal{F}}$; for the plain budget S is a scalar.

Nearly dependent rows. With $1 < m_c \le n_{\mathcal{F}}$ the rows are first replaced by an orthonormal basis of their span, $C_{\mathcal{F}}^{\mathsf{T}} = QR$, and the system is solved with Q in place of $C_{\mathcal{F}}^{\mathsf{T}}$, the target $R^{-\mathsf{T}} r_2$, and the multipliers recovered by a triangular solve with R. Since $\operatorname{cond}(X^{\mathsf{T}} B X) \le \operatorname{cond}(B)\operatorname{cond}(X)^2$ for X of full column rank and $B \succ 0$, with equality possible, the condition number of $C_{\mathcal{F}} \Sigma_{\mathcal{F}\mathcal{F}}^{-1} C_{\mathcal{F}}^{\mathsf{T}}$ can reach $\operatorname{cond}(\Sigma_{\mathcal{F}\mathcal{F}})\operatorname{cond}(C_{\mathcal{F}})^2$, while $Q^{\mathsf{T}} \Sigma_{\mathcal{F}\mathcal{F}}^{-1} Q$ is no worse conditioned than $\Sigma_{\mathcal{F}\mathcal{F}}$, so nearly dependent active rows lose accuracy in proportion to $\operatorname{cond}(C_{\mathcal{F}})$, the sensitivity of the constraint data, and not to its square. Solving only for the free block keeps every step a dense solve of size $|\mathcal{F}|$, where Markowitz *et al.* (2021) assemble one large KKT system over all variables.

Reduced gradients. To decide when a blocked asset should re-enter, the code forms the reduced gradients

$$\gamma = \Sigma\,\alpha + C^{\mathsf{T}} \nu^{\alpha}, \qquad \delta = \Sigma\,\beta + C^{\mathsf{T}} \nu^{\beta} - \mu, \tag{7}$$

where ν stacks the equality and active-row multipliers, so that $\gamma_i + \lambda \delta_i$ is (proportional to) the bound multiplier of a blocked asset i along the segment; the products $\Sigma\alpha$, $\Sigma\beta$ go through the covariance operator of Section 5.

4.2. Event detection

The segment stays optimal until one of four events changes the active set. Two act on variables through the box bounds:

- **Free → blocked.** A free weight $w_i = \alpha_i + \lambda\beta_i$ meets its upper bound at $\lambda_i = (u_i - \alpha_i)/\beta_i$ when $\beta_i < 0$, or its lower bound at $\lambda_i = (\ell_i - \alpha_i)/\beta_i$ when $\beta_i > 0$.
- **Blocked → free.** A blocked asset's multiplier changes sign at $\lambda_i = -\gamma_i/\delta_i$, admissible at the upper bound with $\delta_i < 0$ or the lower bound with $\delta_i > 0$.

The other two are the same moves on the rows of $Gw \le h$, described in Section 4.3. The candidates form a matrix $L \in \mathbb{R}^{(n+p)\times 4}$, one row per variable or inequality row, with $-\infty$ where an event cannot occur, under two numerical guards:

- **Slope floor.** A slope at round-off level is treated as zero. The floor is $\sqrt{\varepsilon_{\text{mach}}}$ in the problem's own units, so the trace is invariant to rescaling μ or Σ (Appendix A.1).
- **λ window.** Candidates above the current λ, beyond a relative 10^{-10}, lie outside the segment and are discarded; the same window decides which events are tied.

The next turning point is $\lambda^\star = \max_{i,t} L_{it}$; if $\lambda^\star \le 0$ the frontier is complete.

4.3. General linear inequalities

A row of $Gw \le h$ is the analogue of a box bound: either *active* ($g_j^\mathsf{T} w = h_j$) or slack. While active it is carried through the reduced solve as an extra equality row, stacked under A into $C_\mathcal{F} = [\, A_\mathcal{F}; G_{\mathcal{S}\mathcal{F}} \,]$, so each step is still one solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ (through the operator interface) feeding an $(m + |\mathcal{S}|)$-dimensional Schur complement. The box bounds stay a separate per-variable active set, so the box-only problem pays nothing for the generalization.

The stacked solve (5) returns an affine multiplier $\eta_j(\lambda) = \eta_j^\alpha + \lambda\,\eta_j^\beta$ for each active row, and the two row events mirror the box events:

- **Activate.** An inactive row with slack $s_j(\lambda) = (g_j^\mathsf{T}\alpha - h_j) + \lambda\, g_j^\mathsf{T}\beta$ binds at $\lambda_j = (h_j - g_j^\mathsf{T}\alpha)/(g_j^\mathsf{T}\beta)$, admissible when $g_j^\mathsf{T}\beta < 0$ (the slack rises as λ falls).
- **Release.** An active row releases at $\lambda_j = -\eta_j^\alpha/\eta_j^\beta$, where its multiplier reaches zero, admissible when $\eta_j^\beta > 0$.

These fill columns 0 and 1 of the trailing p rows of L, whose columns 2 and 3 are $-\infty$, so a group cap or factor-exposure band is traced along the whole frontier (Section 7.1). Each active row adds a column to the solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ and a row to the Schur complement; with many active rows that term, not the covariance solve, dominates a step. A row that is tight but a combination of the active rows on the free set is supported: at the first vertex only an independent subset of the tight rows is made active (Section 3), and during the trace such a row has zero slope, $g_j^\mathsf{T}\beta = 0$ (since $C_\mathcal{F}\beta_\mathcal{F} = 0$), so its activation event does not fire. The active rows themselves must stay linearly independent on the free set, or the Schur complement is singular, as when several caps bind at once in the same direction. Before each solve the rows are tested, and a dependent set is refused with a `DegenerateProblemError` at that λ.

4.4. Gross-exposure caps

A cap $\|w\|_1 \le c$ on gross exposure (under a full budget, $c = 1.3$ is a 130/30 book) is polyhedral but not a single row of G. The code lifts it. Each asset whose box straddles zero is split into

a long and a short leg, $w_i = x_i^+ - x_i^-$ with $x_i^+ \in [0, u_i]$ and $x_i^- \in [0, -\ell_i]$; a long-only or short-only asset keeps one leg, so a long-only problem is not enlarged. With $w = Px$ for the signed leg-to-asset map, the cap becomes the one row $\mathbf{1}^\mathsf{T} x \le c$ appended to GP, the other constraints carry over as AP and GP, and the mean becomes $P^\mathsf{T}\mu$. The lifted program is an instance of (1) with the same λ, traced by the loop above and mapped back through P (validated in Section 7.1).

The lifted covariance $P^\mathsf{T}\Sigma P$ is singular, since raising both legs of one asset leaves w unchanged. The loop never meets that singularity: both legs of an asset are never free together, so every free block is a signed principal submatrix of Σ, served through the operator interface without forming $P^\mathsf{T}\Sigma P$, and a factor backend keeps its Woodbury solve. Appendix A.2 gives the argument and two edge cases of the first vertex.

4.5. Anti-cycling and termination

On degenerate problems (tied returns, exchangeable assets, several constraints active at once) several events can share $\lambda^\star$, and a naive rule can cycle. Among all events within tolerance of $\lambda^\star$ the implementation picks the lowest coordinate index, and within it the lowest event column, and flips that coordinate alone. Call a partition (free set, blocked bounds, active rows $\mathcal{S}$) *valid* at λ when its segment respects the bounds and the inactive rows there and its multipliers have the right signs.

Proposition 1. *Let $\Sigma \succ 0$, and suppose that at every turning point the constraints holding with equality (the rows of A, the rows of G with $g_j^\mathsf{T} w = h_j$, and the bounds on which a weight sits) have linearly independent gradients, the linear independence constraint qualification over all active constraints, bounds included. Then, in exact arithmetic, the walk never returns to a partition it has left, and it reaches $\lambda = 0$ after finitely many turning points.*

Outline. A partition stays valid while finitely many conditions, each affine in λ, hold; once one fails below a turning point it fails for every smaller λ, so a partition never recurs at a later λ, and without ties every step uses a new one. At a tie, the slope of the next segment solves a strictly convex QP whose KKT system is a linear complementarity problem with a positive-definite matrix, and the implementation's lowest-index single flip is Murty's least-index principal pivot on it, which is finite. Appendix C gives the full argument, including the correspondence between the event types of the code and the complementary pairs of the problem. □

Beyond the hypotheses. The proposition holds in exact arithmetic and under its hypotheses. Table 2 sorts the features of the package by what is proved for them. Two weaker statements hold more widely. Because $\Sigma \succ 0$ the optimiser $w(\lambda)$ is unique, and for a fixed set of active constraints it is affine in λ on the interval where that set is optimal; there are finitely many such sets, so the frontier has finitely many turning points whether or not the constraints are independent. And an iteration cap of $100(n + p + 1)$ stops every trace. A walk in which every coordinate changes status once takes $n + p + 1$ steps, and the cap allows a hundred times that; a trace that reaches it raises an error and never returns a partial frontier. Under the gross-exposure lift n counts the legs and p includes the cap row. No trace in Appendix B used more than 1.5% of it. Outside the hypotheses, then, what is left unproved is only that the walk leaves every tie without revisiting a partition.

Feature	Outside the hypotheses because	What holds
Box bounds, $Aw = b$, $Gw \leq h$, $\Sigma \succ 0$	– (if the active constraints are independent at every turning point)	Proposition 1
Degenerate maximum-return vertex (Section 3)	only the start, which has dependent active constraints	Proposition 1 from the first turning point, if its hypotheses hold there
Tied maximum returns (Section 3)	only the start, resolved on a face of optima	as above
Dependent constraints at a later tie	M of Appendix C only positive semidefinite	finite frontier; cap
Singular Σ, solved on the null space (Appendix B)	$\Sigma \not\succ 0$	cap
Gross-exposure cap (Section 4.4)	the lifted covariance is singular	finite frontier in w; cap

Table 2: What is proved for each feature. The start, at $\lambda = \infty$, is not a turning point: Proposition 1 covers the walk that follows it. "Finite frontier" is the finite number of turning points; "cap" is the iteration cap, which stops every trace with an error rather than a partial frontier. A partition with dependent active rows is refused (Section 4.3).

In floating point. The tie window and the slope floor are meant to make the computed signs agree with the exact ones; that is not proved either. At run time every turning point is validated against the bounds, $Aw = b$ and $Gw \leq h$ before it is accepted, and the KKT certificate of Section 7.1 checks the traces of Table 5 and Appendix B for optimality. Appendix B lists which test families satisfy the hypotheses.

When no admissible event has a positive ratio the walk has reached the minimum-variance portfolio and appends a final turning point $w = \alpha$ at $\lambda = 0$.

Algorithm 1. Critical Line Algorithm (as implemented in **cvxcla**).

Input: μ, Σ, ℓ, u, A, b, G, h, tolerance τ.

1. Compute the maximum-return vertex (Section 3); record its free set and active rows $\mathcal{S}$ with $\lambda \leftarrow \infty$.
2. **While** $\lambda > 0$:
   (a) Fix $w_{\mathcal{B}}$ at the active bounds and stack $C_{\mathcal{F}} = [\, A_{\mathcal{F}}; G_{\mathcal{SF}} \,]$.
   (b) Solve against $\Sigma_{\mathcal{FF}}$ for Y, z^{α}, z^{β}; solve the Schur system (5); back-substitute (6) for α, β; form γ, δ (7).
   (c) Assemble L (Section 4.2, Section 4.3) under the slope floor and λ window; $\lambda^{\star} \leftarrow \max L$; **if** $\lambda^{\star} \leq 0$ **break**.
   (d) Pick the event by the least-index rule (Section 4.5), flip that coordinate's status, set $\lambda \leftarrow \lambda^{\star}$, append $w = \alpha + \lambda^{\star}\beta$.
3. Append $w = \alpha$ at $\lambda = 0$ and **return** the turning points.

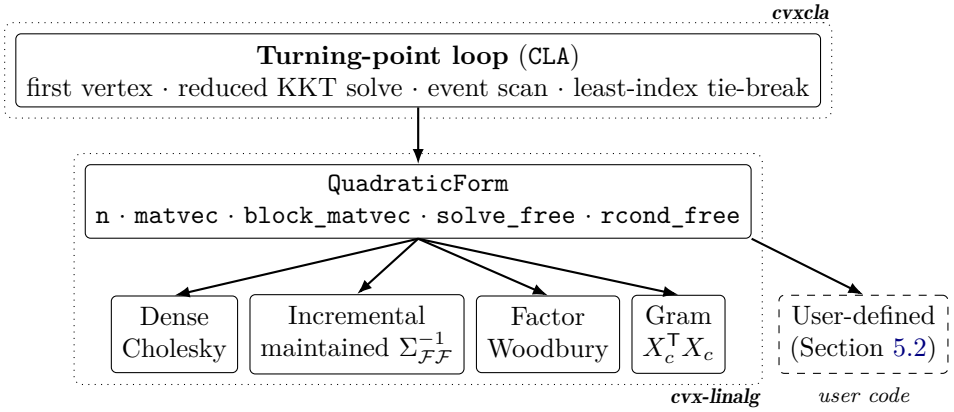


Figure 2: The covariance enters the turning-point loop only through the `QuadraticForm` protocol; the backends below it are interchangeable. Dotted boxes mark the packages: the loop is **cvxcla**'s, the protocol and the four built-in operators are **cvx-linalg**'s, and **cvxcla** builds the operators from its inputs. A user-defined backend implements the protocol in the user's own code.

5. The covariance as an operator

The design separates the path-following algorithm from the storage and solution of the quadratic form (Figure 2). The turning-point loop needs only three operations on the covariance: a matrix–vector product Σx, a multi-column solve against the free block $\Sigma_{\mathcal{F}\mathcal{F}}^{-1} R$, and the free-to-blocked cross product $\Sigma_{\mathcal{F}\mathcal{B}}\, x_{\mathcal{B}}$, which is formed from the first. A backend supplies them through the protocol `QuadraticForm`, defined in the package's linear-algebra dependency **cvx-linalg** (Schmelzer 2026c), whose five members are listed in Table 3; two of them serve the gross-exposure lift and the singularity guard. The algorithm never assumes an explicit matrix, and it assumes of a backend only that Σ is symmetric and that every free block it solves is positive definite on the null space of the active rows, which `rcond_free` lets it check before trusting the solve (Appendix B). Four backends ship with the package, and a user can add others without touching the loop (Section 5.2).

The work is split between the two packages. **cvx-linalg** implements the protocol and the linear algebra behind it: the four operators and their free-block solves (`dense.py`, `factor.py`, `gram.py`), the bordered solve of (4) (`bordered.py`) and the affine projection of Appendix B (`projection.py`). **cvxcla** implements everything that is specific to the frontier: the turning-point loop, the starting portfolio, the event scan and tie-break, the gross-exposure lift, the feasibility checks and the error handling. Its four backends, `DenseCovariance` and the three of the paragraphs below, validate their inputs and return the corresponding **cvx-linalg** operator (`operators/builders.py`).

Dense. `DenseCovariance` wraps an explicit matrix and solves the free block by Cholesky (`scipy.linalg.cho_factor`), falling back to LU on the rare block that is not numerically positive definite. It reproduces the classical CLA and is the reference backend.

Incremental dense. `IncrementalDenseCovariance` traces the same turning points but *maintains* $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$: because the free set changes by one asset per turning point, a bordered

Member	Returns	Used for
`n`	the number of assets	sizes
`matvec(x)`	Σx, for a vector or a matrix x	gradients, the cross product
`block_matvec(rows, cols, v)`	$\Sigma_{\text{rows,cols}}\, v$	the gross-exposure lift (Section 4.4)
`solve_free(free, R)`	$\Sigma_{\mathcal{F}\mathcal{F}}^{-1} R$	the reduced KKT solve (Section 4.1)
`rcond_free(free)`	the reciprocal condition number of $\Sigma_{\mathcal{F}\mathcal{F}}$	the singularity guard

Table 3: The `QuadraticForm` protocol. Index sets are integer arrays into $\{0, \ldots, n-1\}$.

rank-one update or a symmetric deletion costs $O(n_{\mathcal{F}}^2)$ against a fresh $O(n_{\mathcal{F}}^3)$ factorization; Section 7.3 measures the gain. A non-positive Schur pivot or a multi-asset change triggers a full refactorization. The backend passes the same KKT certificate as the dense one (Table 5), but a maintained inverse carries round-off from step to step over the roughly n updates of a trace, so it is opt-in. On the long-only problems of Section 7.2 the drift is small: the incremental weights stay within 2.1×10^{-13} of the dense ones up to $n = 2560$, at the same turning points, and the KKT residual grows from 1.5×10^{-13} to 3.4×10^{-13} (`make_figures.py validate-incremental`).

Factor (diagonal-plus-low-rank). `FactorCovariance` represents

$$\Sigma = \operatorname{diag}(d) + U\,\Delta\,U^{\mathsf{T}}, \qquad d \in \mathbb{R}^n_{>0},\ U \in \mathbb{R}^{n\times K},\ \Delta \in \mathbb{R}^{K\times K}, \tag{8}$$

the form of factor models (Sharpe 1963) and of eigenvalue-clipped covariances. The free-block solve never forms $\Sigma_{\mathcal{F}\mathcal{F}}$; it uses the Woodbury identity

$$\Sigma_{\mathcal{F}\mathcal{F}}^{-1} = D_{\mathcal{F}}^{-1} - D_{\mathcal{F}}^{-1} U_{\mathcal{F}}\, W^{-1}\, U_{\mathcal{F}}^{\mathsf{T}} D_{\mathcal{F}}^{-1}, \qquad W = \Delta^{-1} + U_{\mathcal{F}}^{\mathsf{T}} D_{\mathcal{F}}^{-1} U_{\mathcal{F}} \in \mathbb{R}^{K\times K}, \tag{9}$$

so a solve costs $O(n_{\mathcal{F}} K^2 + K^3)$ rather than $O(n_{\mathcal{F}}^3)$, storage is $O(nK)$, and the cross product is $U_{\mathcal{F}}\,\Delta\,(U_{\mathcal{B}}^{\mathsf{T}} x_{\mathcal{B}})$.

The admissible model is $d > 0$ and Δ symmetric positive semidefinite. `FactorCovariance` folds Δ into the loadings when it is built: with $\Delta = V\Lambda V^{\mathsf{T}}$ it stores $U' = UV_+\Lambda_+^{1/2}$, the directions of positive variance, and an identity inner matrix, which is the same Σ. The Woodbury solve (9) then runs with U' and $W = I + U_{\mathcal{F}}'^{\mathsf{T}} D_{\mathcal{F}}^{-1} U_{\mathcal{F}}'$ and never inverts Δ. A singular factor covariance, from a factor with zero variance or perfectly correlated factors, is therefore admissible, and its rank K is the number of directions kept. An indefinite Δ is refused: a signed low-rank term can still give a positive-definite Σ, but W would then be indefinite, so such a covariance is passed as a dense matrix. Since $d > 0$, W is positive definite, and the factor solve needs no guard of its own beyond the one on $\Sigma_{\mathcal{F}\mathcal{F}}$.

The factor backend was tested against the dense trace of the same Σ on ill-conditioned models: Δ with condition numbers up to 10^{14}, loadings collinear to within 10^{-10}, and the full rank $K = n = 120$ (`make_figures.py validate-factor`). It finds the same turning points, with weights equal to within 1.0×10^{-12} and KKT residuals of at most 1.0×10^{-13}. A low-rank *approximation* of a sample covariance is a different matter: it traces the frontier of the approximation, which is a different frontier (Section 7.7).

Data matrix (Gram). A sample covariance is a Gram matrix: for the column-centred returns $X_c \in \mathbb{R}^{T\times n}$, $\Sigma = X_c^{\mathsf{T}} X_c/(T-1)$. `GramCovariance` implements the protocol straight from X_c in $O(Tn)$ storage. When $T < n$ the matrix is singular, and once the free set outgrows the data rank the free block is singular too; the active rows still hold it on their null space, where it is solved (Appendix B); Proposition 1 assumes $\Sigma \succ 0$, so for such a trace only the iteration cap guarantees termination (Table 2). An optional ridge θI (the argument `ridge`) makes Σ positive definite, a different, regularized frontier, and the solve is then Woodbury in the T-dimensional observation space. The saving is in storage: when the dense matrix fits, `DenseCovariance` is as fast or faster, and the structured route pays only while T (or K) stays well below n (Section 7.4). On the last 300 days of the S&P 500 data of Section 7.7 ($T = 300 < n = 494$) both backends trace the same 81 turning points, with weights within 2×10^{-14}; the Gram backend takes 41 ms against 30 ms for the dense one without a ridge, and 22 ms for both with a ridge of 10^{-5} (`make_figures.py real-frontier`). Both traces pass the KKT certificate (Table 5).

5.1. Regularization through the operator

A quadratic penalty $\frac{\theta}{2} w^{\mathsf{T}} M w$, with $\theta \geq 0$ and M symmetric positive semidefinite and independent of λ, changes only the Hessian, so the path stays piecewise linear and the penalty is the substitution $\Sigma \mapsto \Sigma + \theta M$: a ridge ($M = I$) or a factor-exposure penalty ($M = BB^{\mathsf{T}}$ for exposures B). `CLA` has no penalty argument; the regularized form is supplied as the covariance, a bundled or a user-written `QuadraticForm` (Section 5.2). `GramCovariance` with $\theta > 0$ and `FactorCovariance` already are such forms. A rank-deficient sample covariance needs neither: its frontier is traced as it is, as long as the active rows keep the free block positive definite on their null space (Appendix B); a small ridge or a factor model is the fix when they do not.

5.2. Writing a backend

A backend is a subclass of `QuadraticForm` that implements the five members of Table 3. For a block-diagonal covariance, assets in groups with no cross-group covariance (a structure the package does not ship), the free-block solve splits into one Cholesky solve per block. The two members that carry the structure are below; `n`, `matvec` and `block_matvec` are a few lines each. The complete class, with the check below, is `custom_backend.py` in the replication materials and is also in the package documentation (`docs/custom-backend.md`).

```
class BlockDiagonalCovariance(QuadraticForm):  # blockdiag(S_1, ..., S_m)
    def solve_free(self, free, rhs):
        out = np.empty_like(rhs, dtype=float)
        # one (positions, free sub-block) pair per group
        for pos, sub in self._free_blocks(free):
            out[pos] = cho_solve(cho_factor(sub), rhs[pos])
        return out

    def rcond_free(self, free):
        eig = np.concatenate(
            [np.linalg.eigvalsh(sub) for _, sub in self._free_blocks(free)]
```

```
    )
    return eig.min() / eig.max()
```

`CLA` runs on it unchanged: on four groups of five assets it traces the same 19 turning points as the dense matrix. The package's test suite, `tests/test_custom_backend.py`, checks the same with general constraints and with a gross-exposure cap, which goes through `block_matvec`. The frontier is exact to the accuracy of `solve_free`; an approximate solver traces the frontier of the matrix it effectively applies. The package checks each turning point for feasibility only; optimality against the intended Σ is what the KKT certificate of Section 7.1 tests. In release 2.7.1 the certificate is a function of the replication script (`_kkt_check` in `make_figures.py`), not of the package: it applies to any trace, but it needs the dense Σ, which a structured backend exists to avoid, and a linear program at vertices whose multipliers are not determined. A user who can afford the dense matrix can run it on their own trace; moving it into the package is left to a later release. A sparse matrix with a sparse Cholesky factor, a Kronecker product, or a matrix-free operator with an iterative `solve_free` fit the same five members.

6. Using the package

The package (Schmelzer 2026d) is installed with `pip install cvxcla`; it is pure Python and needs **NumPy**, **SciPy** and **cvx-linalg** (Computational details). The workflow goes through a single class, `CLA`. The user supplies μ, the covariance (a dense array or any backend of Section 5), the bounds, and the constraints; constructing the object traces the whole frontier. The example traces a small long-only, fully-invested problem and reads off the maximum-Sharpe portfolio.

```
import numpy as np
from cvxcla import CLA, FactorCovariance

rng = np.random.default_rng(1)
n, k = 6, 2
d = rng.uniform(0.1, 0.5, n)            # idiosyncratic variances
U = rng.standard_normal((n, k))         # factor loadings
Delta = np.diag(rng.uniform(0.5, 2.0, k))
Sigma = np.diag(d) + U @ Delta @ U.T    # a dense 2-factor covariance
mean = rng.standard_normal(n)

cla = CLA(
    mean=mean, covariance=Sigma,
    lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
    a=np.ones((1, n)), b=np.ones(1),    # fully invested: sum(w) = 1
)
print("turning points:", len(cla.turning_points))
sr, w = cla.frontier.max_sharpe
print("max Sharpe:", round(sr, 4))
print("weights:", np.round(w, 3))
```

```
turning points: 7
max Sharpe: 2.4037
weights: [0.    0.    0.704 0.296 0.    0.   ]
```

Switching to a structured backend changes only the covariance argument; because the factor backend represents the *same* Σ, it returns the same frontier to round-off.

```
factor = FactorCovariance(d=d, u=U, delta=Delta)
cla_factor = CLA(
    mean=mean, covariance=factor,
    lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
    a=np.ones((1, n)), b=np.ones(1),
)
w_dense = np.array([t.weights for t in cla.turning_points])
w_factor = np.array([t.weights for t in cla_factor.turning_points])
print("identical frontier:", np.allclose(w_dense, w_factor))
```

```
identical frontier: True
```

A fluent builder, `CLA.problem`, assembles the same arguments step by step through methods such as `.budget`, `.equality` and `.inequality`; an extra `.equality` row, such as a fixed sector weight, makes the first turning point the linear program of Section 3.

6.1. The interface

`CLA` takes the problem as keyword arguments, all **NumPy** arrays of floats, and traces the frontier in its constructor, so every `CLA` object holds a complete, validated frontier. Its input arrays are copied read-only (see below), and a changed μ or Σ needs a new object and a full trace: the package does not warm-start a trace from an earlier one, although the old sequence of active sets would be a natural starting guess. Sensitivity studies, resampling and rolling estimation, which re-trace after small changes in μ, therefore pay the full cost each time. Required are `mean` (μ, length n), `covariance` (an $n \times n$ array, which is wrapped as a `DenseCovariance`, or any backend of Section 5), `lower_bounds` and `upper_bounds` (length n), and `a`, `b` ($m \times n$ and length m; a budget is `a = np.ones((1, n))`, `b = np.ones(1)`). Optional are `g`, `h` ($p \times n$ and length p) for $Gw \leq h$, `leverage` for the cap $\|w\|_1 \leq c$, and `tol` (default 10^{-5}), the tolerance for deciding that a weight sits on a bound at the first vertex and for validating each turning point: a point that violates a bound, a row of G or the cap by up to `tol` after the projection of Appendix B is accepted, while $Aw = b$ is checked to 10^{-7} regardless (Appendix A.1). The tolerance is a ceiling: the projection itself stops at 10^{-12}, and on the traces of Table 5 no tested point violates a constraint by more than 1.1×10^{-13}. The default is loose for a reason: at a linear-programming first vertex `tol` decides which weights **HiGHS** has put on a bound, and **HiGHS** meets its constraints only to its own feasibility tolerance, 10^{-7} by default, so `tol` must stay well above that. On the problems of Appendix A.1 every value from 10^{-4} to 10^{-10} gives the same traces. The event ordering uses its own relative window of at most 10^{-10} (Section 4.2). The arrays (and a dense covariance) are copied into read-only arrays when the object is built, so changing the caller's arrays afterwards cannot put the stored turning points out of step with the data; a backend is used as given. Non-finite

data are refused at this point, as are NaN bounds. Redundant equality rows are dropped, and `cla.a`, `cla.b` then hold the reduced system.

The result is `cla.turning_points`, a list of `TurningPoint` objects with the `weights`, the value `lamb` of λ, the boolean mask `free` of free assets, and the mask `active_ineq` of active rows of G. The maximum-return vertex appears twice, at $\lambda = \infty$ and again at the first finite turning point, where the path leaves it, so that every pair of consecutive turning points bounds one segment of the path, the first being the constant stretch above the first finite λ; `len(cla)` is the number of turning points, which counts that vertex twice, so the number of distinct portfolios is `len(cla) - 1`. Software that lists each turning point once, such as the R package **CLA**, reports one fewer (Section 7.1). `cla.frontier` wraps them as a `Frontier` (`types.py`), which exposes the `weights`, `returns`, `variance`, `volatility` and `sharpe_ratio` of the turning points and a `plot` method. Because the frontier is piecewise linear in weight space, `interpolate(num)` returns intermediate portfolios with no discretization error.

6.2. The frontier and the maximum-Sharpe portfolio

`max_sharpe` maximizes $\mu^\mathsf{T} w / \sqrt{w^\mathsf{T} \Sigma w}$, the Sharpe ratio with a zero risk-free rate (supply excess returns as μ otherwise), over the computed frontier. On a segment $w(t) = (1-t)\, w_0 + t\, w_1$, $t \in [0, 1]$, the return is affine and the variance quadratic in t, so the Sharpe ratio $S(t)$ has at most one stationary point,

$$t^\star = \frac{a_0 c_1 - 2 a_1 c_0}{a_1 c_1 - 2 a_0 c_2}, \tag{10}$$

with $a_0 = \mu^\mathsf{T} w_0$, $a_1 = \mu^\mathsf{T}(w_1 - w_0)$ and $c_0 + c_1 t + c_2 t^2$ the variance along the segment. The code compares $S(0)$, $S(1)$ and, when $0 < t^\star < 1$, $S(t^\star)$ on every segment and returns the best, the exact maximum over the computed frontier, without assuming the ratio is unimodal, at the cost of one matrix–vector product with Σ per segment.

Points of zero variance are skipped (`sharpe_ratio` reports NaN there); with no positive variance `max_sharpe` raises a `ValueError`, and with no positive expected return it warns and returns the least negative ratio. Whether the ratio, computed from estimated μ and Σ, predicts future performance is a statistical question outside the package.

6.3. Supported inputs and errors

Table 4 sorts the inputs into kinds, each with its own exception and remedy, and each row is a test in the package (`tests/test_errors.py` and `tests/test_degenerate_vertex.py`). All are raised by the constructor. The package errors derive from `CLAError` and also from the built-in exception raised before (`ValueError`, or `RuntimeError`), so existing handlers keep working.

A sample covariance from fewer observations than assets is supported. It is singular, and its free block becomes singular partway along the frontier. The budget row still holds the free assets on its null space, where the block stays positive definite, so the trace goes on there (Appendix B). Termination is then guaranteed by the iteration cap, not by Proposition 1, which assumes $\Sigma \succ 0$. A ridged Gram backend (Section 5) traces a different, regularized frontier:

Input	Remedy
Malformed input: `ValueError`, before the walk	
Non-symmetric Σ; non-finite data or NaN bounds; an indefinite factor Δ; wrong shapes	correct the data
Infeasible: `InfeasibleProblemError`, before the walk	
Crossed bounds; an unreachable budget; constraints with no feasible point; dependent equality rows that contradict the others	change the constraints
Harmless degeneracy: traced, nothing raised	
Redundant equality rows; duplicated, never-binding or locally redundant inequality rows; a degenerate maximum-return vertex whose optimal dual set has a vertex, and tied maximum returns (Section 3); a singular covariance whose free blocks the active rows keep positive definite (Appendix B)	–
Unsupported degeneracy: `DegenerateProblemError`	
Before the walk: a degenerate vertex whose optimal dual set has no vertex, as when a tight row touches only assets fixed at both bounds; infinite bounds that make the return unbounded	perturb the bounds or constraints
During the walk: a free block singular even on the null space of the active rows, or active rows dependent on the free set	a ridge or factor model; loosen a cap
Numerical failure: `NumericalError`, during the walk	
Iteration cap; projection fails; a turning point violates a constraint	regularize Σ or relax `tol`; report it

Table 4: Supported and unsupported inputs, grouped by the exception each raises and when, with the usual remedy. "Before the walk" is input validation and the first vertex; "during the walk" is a turning point of the trace.

```
from cvxcla import GramCovariance

returns = 0.01 * rng.standard_normal((4, n)) + 0.001  # 4 days: rank 3
long_only = dict(lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
                 a=np.ones((1, n)), b=np.ones(1))
cla = CLA(mean=returns.mean(axis=0),
          covariance=np.cov(returns, rowvar=False), **long_only)
ridged = CLA(mean=returns.mean(axis=0),
             covariance=GramCovariance(returns, ridge=1e-4), **long_only)
print("turning points:", len(cla), "| with a ridge:", len(ridged))
```

```
turning points: 5 | with a ridge: 7
```

Every turning point of the trace above matches an interior-point solve of the same quadratic program. With a singular Σ the optimal weights at a given λ need not be unique, although their return and variance are: **cvxcla** returns the optimal portfolio its active set determines, and another solver may return a different one (Appendix B).

6.4. A worked example: a 130/30 book

The example puts the pieces together on the S&P 500 snapshot of Section 7.7, 494 stocks over 1213 days; the path to the data is relative to the replication materials. The book is fully invested, holds at most 5% and is short at most 3% of any stock, keeps ten large technology names at 6% in total, and caps gross exposure at 130%. The builder `CLA.problem` states the problem term by term; `.inequality` adds the group cap as one row of $Gw \leq h$ and `.leverage` the gross cap, which **cvxcla** lifts (Section 4.4).

```
import numpy as np
import pandas as pd
from cvxcla import CLA

returns = pd.read_parquet("data/sp500_pct_returns.parquet")
mu = 252 * returns.mean().to_numpy()            # annualized moments
Sigma = 252 * returns.cov().to_numpy()

tech = ["AAPL", "MSFT", "NVDA", "GOOGL", "META",
        "AMZN", "AVGO", "ORCL", "ADBE", "CRM"]
in_tech = returns.columns.isin(tech).astype(float)
```

The book is then stated and traced in one expression:

```
book = (
    CLA.problem(mu, Sigma)
    .bounds(-0.03, 0.05)                # at most 3% short, 5% long a stock
    .budget()                           # fully invested
    .inequality(in_tech[None, :], 0.06) # the ten names: at most 6% in all
    .leverage(1.3)                      # gross exposure at most 130%
    .trace()
)
binds = sum(tp.active_ineq[0] for tp in book.turning_points)
gross = sum(abs(tp.weights).sum() > 1.3 - 1e-9 for tp in book.turning_points)
print(len(book), "turning points; the cap binds at", binds,
      "and the gross cap at", gross)
_, w = book.frontier.max_sharpe
print(f"max Sharpe: gross {abs(w).sum():.2f}, the ten names {in_tech @ w:.3f},",
      (w > 1e-9).sum(), "long,", (w < -1e-9).sum(), "short")
```

```
250 turning points; the cap binds at 95 and the gross cap at 250
max Sharpe: gross 1.30, the ten names 0.044, 47 long, 9 short
```

The group cap binds over part of the frontier and is released elsewhere, both events of Section 4.3; the gross cap binds at every turning point. The data carry the survivorship bias of Section 7.7, so the maximum-Sharpe book illustrates the interface, not an investment. Between turning points the frontier is exact, so `interpolate` fills it in without re-solving, and `book.frontier.plot()` draws it (with **plotly**, an optional dependency). Figure 3 sets the

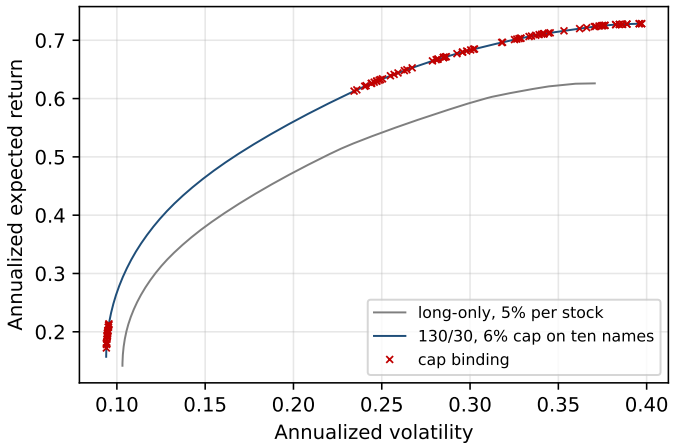


Figure 3: The frontier of the 130/30 book of Section 6.4 against the long-only book with the same 5% limit per stock, both drawn with `interpolate`. Crosses mark the turning points at which the 6% cap on the ten technology names binds (`make_figures.py worked-example`).

book against the long-only one with the same 5% limit, traced by `.bounds(0.0, 0.05)` in place of the last three constraints. A problem with no feasible point is refused before the walk starts, with the exception of Table 4:

```
fine = book.frontier.interpolate(20)     # 20 portfolios per segment
print(len(fine.weights), "portfolios on the exact frontier")

from cvxcla import InfeasibleProblemError
try:
    CLA.problem(mu, Sigma).bounds(0.0, 0.001).budget().trace()
except InfeasibleProblemError as err:
    print(type(err).__name__ + ":", err)
```

```
4731 portfolios on the exact frontier
InfeasibleProblemError: Could not construct a fully invested portfolio
```

The whole example, the figure included, is one step of the replication script.

7. Numerical experiment

The experiments answer three questions, in this order: whether the trace is correct (Section 7.1); whether its speed on large problems comes from the structured solve rather than from bookkeeping (Sections 7.2–7.4); and how it compares with the alternatives, a QP swept over a grid and a parametric QP solver (Sections 7.5 and 7.6). A real S&P 500 frontier closes the section (Section 7.7); further checks are in Appendices A and B. Timings are the median of eleven repetitions up to $n = 160$, where traces are short, and of three above, the 5-minute **qpOASES** runs at $n = 2560$ included (bands span their range), each method and size in a

freshly spawned interpreter, on the machine of Computational details. All methods run with their default threading: the dense backends call a multithreaded BLAS, **OSQP** and **Clarabel** solve on one thread, and the points of a grid are solved one after another. Restricting **cvxcla** to one BLAS thread as well (`make_figures.py threads`) changes its times at $n = 640$ by at most 12%, so the ratios to the grids there stand; at $n = 2560$ it leaves the factor backend unchanged and slows the dense backend by 1.26× and the incremental one by 1.52×, so on one thread each the factor backend's advantage over them is larger than reported below (40× over the dense backend). Within a run, the repetitions above $n = 160$ agree to within a few percent (**qpOASES** at $n = 2560$ to 14%). Across three complete runs on different days the medians moved more: by up to 26% for **cvxcla**'s backends (the dense trace at $n = 1280$ took 5.5, 4.7 and 4.3 s) and by up to 74% for the external solvers (**qpOASES** at $n = 2560$ took 548, 416 and 314 s, **Clarabel**'s grid 3.0, 1.9 and 1.8 s). We report the third run, which Figure 4 shows, and give the range over the three where a ratio the text relies on moved; the ratios are indicative to that extent, and those at small n only roughly.

7.1. Correctness: a reference solver and a KKT certificate

On 40 real assets (Section 7.7; condition number ≈ 200) the CLA traces 29 turning points. At the midpoint λ of each of the 27 interior segments we re-solve (1) with **OSQP** (Stellato, Banjac, Goulart, Bemporad, and Boyd 2020), polished and at tight tolerances, and compare with the interpolated CLA weights: the median discrepancy is 2×10^{-15} and the maximum 6×10^{-8}, solver precision (`make_figures.py validate-exact`). Since each segment is affine, one interior point certifies it. The step `make_figures.py validate-constraints` validates two constrained frontiers the same way on a 30-asset factor market: a budget plus a neutrality row (the LP first vertex) and a budget with per-sector caps (an inequality active at 30 of 31 turning points). Both match to 3×10^{-14} or better. A third frontier is a 130/30 book: the budget, shorts down to -30% per asset, and the gross-exposure cap $\|w\|_1 \leq 1.3$ of Section 4.4, which binds at 32 of 49 turning points; it matches to 1.4×10^{-12}. The reference QP models the cap with auxiliary variables $t \geq |w|$, independently of the lift. An independent implementation of the CLA agrees as well: on the long-only problems of Section 7.2 up to $n = 640$, the R package **CLA** (Shi and Maechler 2024) returns the same turning points as **cvxcla**, with the maximum-return vertex counted once, each within 5×10^{-13} of its counterpart (`make_figures.py r-cla`). It takes 26 s at $n = 320$ and 12 minutes at $n = 640$, so it is not part of the timing comparison of Section 7.2.

A reference solver only shows that two codes agree; a KKT certificate tests the trace itself (`make_figures.py validate-kkt`). At three interior points of every segment, and at two values of λ above the first turning point, the active set is read off the interpolated weights (within 10^{-9} of a bound or of h), the multipliers are solved from stationarity on the free coordinates, and primal feasibility, stationarity, dual feasibility and complementary slackness are each measured. Neither the multipliers nor the partition come from **cvxcla**, and the certificate always uses an explicit dense Σ, never the backend that traced the frontier. Where the free coordinates do not determine the multipliers, at a vertex with no free weight or where two active rows coincide on the free set (the budget and the gross cap do when every free leg is long), stationarity on the free coordinates fixes the multipliers up to a small null space, and a linear program over it asks whether sign-feasible multipliers exist. Primal feasibility is absolute, the other conditions relative to $\max(1, \|\Sigma w\|_\infty, \lambda\|\mu\|_\infty)$. Over 18,830 points on thirteen traces (Table 5), including the incremental, factor and Gram backends (the last on

Problem	cond(Σ)	points	primal	station.	dual	compl.
S&P 500, 40 assets	2.1×10^{2}	83	2×10^{-16}	1×10^{-18}	0	4×10^{-22}
S&P 500, 494 assets	1.1×10^{5}	284	3×10^{-16}	4×10^{-18}	0	5×10^{-27}
incremental backend	1.1×10^{5}	284	7×10^{-16}	4×10^{-18}	0	3×10^{-23}
last 600 days	4.7×10^{5}	335	4×10^{-16}	6×10^{-17}	0	3×10^{-23}
last 300 days, Gram	9.0×10^{18}	239	4×10^{-16}	4×10^{-18}	0	2×10^{-23}
same, ridge 10^{-5}	7.2×10^{3}	239	4×10^{-16}	4×10^{-18}	0	7×10^{-21}
130/30 book (Section 6.4)	1.1×10^{5}	737	4×10^{-14}	6×10^{-16}	0	1×10^{-14}
Factor market, 320 assets	1.3×10^{3}	965	3×10^{-14}	9×10^{-15}	0	0
Factor market, 2560 assets	9.6×10^{3}	7679	9×10^{-14}	3×10^{-14}	2×10^{-16}	7×10^{-20}
dense backend	9.6×10^{3}	7679	1×10^{-13}	2×10^{-14}	1×10^{-16}	1×10^{-25}
30 assets:						
neutrality row	8.3×10^{1}	98	5×10^{-14}	1×10^{-15}	0	2×10^{-15}
sector caps	8.3×10^{1}	89	4×10^{-15}	2×10^{-15}	0	1×10^{-15}
130/30 cap	8.3×10^{1}	119	4×10^{-15}	9×10^{-16}	3×10^{-16}	4×10^{-15}

Table 5: Largest KKT residual of each condition (primal feasibility, stationarity, dual feasibility, complementarity) over all points tested on a trace. Primal feasibility is absolute; the other three are relative to $\max(1, \|\Sigma w\|_\infty, \lambda\|\mu\|_\infty)$. The factor markets are traced with the factor backend, unless marked, and checked against their dense Σ; the 130/30 problems are checked in their lifted long/short form. Produced by `make_figures.py validate-kkt`.

a singular sample covariance, with and without a ridge), the general constraints, the lifted 130/30 problem, the $n = 2560$ market of Section 7.2 under both backends and the worked example of Section 6.4, no residual exceeds 2×10^{-13} on the machine of Computational details. The other sizes of the timing sweeps and the R comparison are not certified. The certificate belongs to the replication script, not to the package: at run time **cvxcla** checks each turning point for feasibility only (Section 4.5). The test does detect errors: weights shifted by 10^{-3} give a residual of 2.5×10^{-7}, and weights paired with the λ of another segment 3.3×10^{-4}.

7.2. Runtime versus problem size

We trace the full frontier for $n \in \{20, 40, \ldots, 2560\}$ assets, each from a ground-truth $K = 10$ factor model, so the dense and factor backends run the same loop on the same Σ and differ only in the free-block solve (`make_figures.py scaling`). The free set is not small: on this market it grows to all n assets, with a median of $n/2$ over the trace. With the roughly n steps observed here, a dense trace is about n solves of $O(n_{\mathcal{F}}^3)$ each, $O(n^4)$ operations, while the per-step cost of the factor backend is $O(n_{\mathcal{F}}K^2 + nK)$, so $O(n^2K)$ in total. At small n a step's fixed overhead hides both rates, and the two backends are at parity up to $n \approx 80$. Over the largest four sizes (320 to 2560) the fitted exponents are ≈ 2.9 for the dense backend and ≈ 1.4 for the factor backend (Figure 4), below the operation counts; we report them as measured and do not extrapolate them. The factor backend is about 3.5× faster at $n = 640$, 10× at 1280 and 33× at $n = 2560$ (32× to 36× over the three runs), where it traces the 2561 turning points in about 1.2 s against 39 s (Figure 4). The ratio depends on the BLAS, and in the factor backend's favour: on Linux with the OpenBLAS bundled with **NumPy**, in a container on the same machine (Computational details), the two traces take 1.5 s and 234 s, about 160× (`make_figures.py blas`), since the dense free-block solves gain far more from Apple's Accelerate than the factor backend's small ones. This market is the favourable case

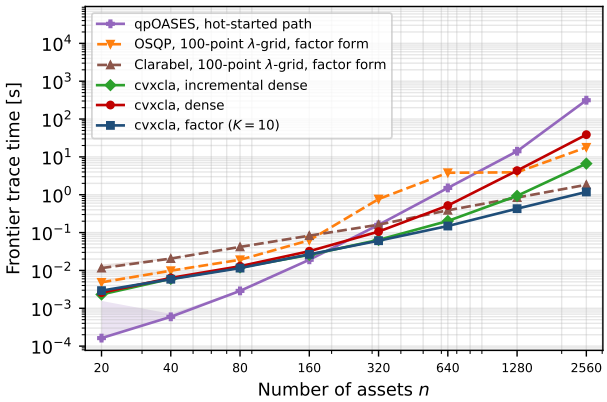


Figure 4: Frontier trace time against n (log–log). Exponents p of time $\approx n^p$, fitted over the largest four sizes ($n = 320$–2560) with no overhead removed: 2.9 dense, 2.2 incremental dense and 1.4 factor, 1.4 **OSQP** and 1.2 **Clarabel** on the factor form, and 3.6 **qpOASES**; they describe these sizes on this machine, not asymptotic complexity. The grids have 100 values of λ (Section 7.5), and **qpOASES** is hot-started along the whole path (Section 7.6). The methods deliver different outputs: **cvxcla** returns every turning point, the grids 100 portfolios each, and **qpOASES** only the minimum-variance endpoint, so its time covers the path following alone.

for the factor backend, since every asset is free at some point. On a rank-20 factor model of the S&P 500 (Section 7.7), where at most 55 of 494 assets are free at once, both backends trace the same 89 turning points, the factor backend about 1.4× faster (18 against 25 ms). On that model's version of the 130/30 book of Section 6.4, whose free set reaches 88 assets, the factor and dense backends take 90 and 96 ms for 238 turning points: there the per-step overhead, not the solve, dominates.

The factor backend also never holds an $n \times n$ matrix. At $n = 2560$ its covariance takes 0.22 MiB against 50 MiB, and the trace's working memory, the `tracemalloc` peak less what the finished trace retains, is 0.84 MiB against 100 MiB (Python objects and **NumPy** buffers; BLAS workspace is not counted). The stored frontier, n weights per turning point, is $O(n^2)$, 50 MiB at $n = 2560$, for either backend.

7.3. Where the speed comes from

To separate the structured solve from the dense linear algebra, we set **cvxcla**'s own dense backend against its opt-in incremental one, `IncrementalDenseCovariance` (Section 5), which runs the same loop but maintains $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$ by rank-one updates and reports the same turning points. A trace is then $O(n^3)$, the measured exponent ≈ 2.2, and the incremental backend is about 1.7× faster than the dense one at $n = 320$, 2.6× at $n = 640$ and 6× at $n = 2560$: block elimination is not the fastest dense strategy. The factor backend is still about 1.4× faster than the incremental one at $n = 640$ and 5.6× at $n = 2560$, so the speed comes from the structured solve. Updating the Woodbury capacitance W incrementally would add little at $K = 10$, where the $O(n)$ work of each step (event scan, products with Σ, bookkeeping) dominates, and matters only at large K, where the dense backend is already faster (Section 7.4).

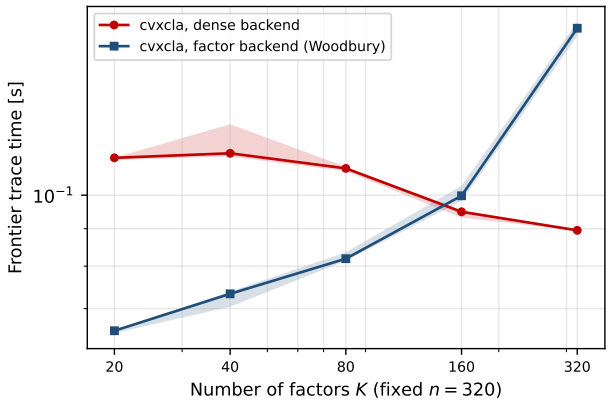


Figure 5: Frontier trace time against the factor rank K at $n = 320$ (log–log). The dense backend does not see K; the Woodbury solve of the factor backend costs $O(n_{\mathcal{F}}K^2 + K^3)$ per step.

7.4. Runtime versus factor rank

The factor backend's advantage must fade as K approaches n. At fixed $n = 320$ and $K \in \{20, \ldots, 320\}$ (Figure 5, `make_figures.py rank-scaling`), the dense time is roughly flat in K while Woodbury grows as $O(n_{\mathcal{F}}K^2 + K^3)$: the factor backend is about 1.7× faster at $K = 20$, at parity near $K = 160$, and $\approx 1.9\times$ slower at $K = 320$. It pays off in the regime real risk models inhabit, a few tens of factors over hundreds or thousands of assets.

7.5. A general-solver baseline: a QP swept over λ

General toolkits (**Riskfolio-Lib** (Cajas 2021), **skfolio** (Nicolini, Manzi, and Delatte 2025), **fPortfolio** (Würtz, Chalabi, Chen, and Ellis 2009), **CVXR** (Fu, Narasimhan, and Boyd 2020)) hand a single risk target to a general solver, so a frontier means re-solving (1) on a grid. We make the grid as favorable as we can: the solver is set up once, each λ changes only the linear term, and the grid has 100 values of λ (`make_figures.py scaling`), equally spaced from the first finite turning point down to 0 on the synthetic markets and geometrically spaced between the largest and smallest positive turning point on the S&P 500. A grid's cost is proportional to its size, while the trace returns all its turning points at once; at $n = 2560$ there are 2561, so a grid as fine as the trace would cost about 25 times as much as the one timed here. We give the solvers the factor model, as the factor backend has it. With $y = U^{\mathsf{T}}w$ as K extra variables, the risk $w^{\mathsf{T}}\Sigma w$ of (8) becomes $w^{\mathsf{T}}Dw + y^{\mathsf{T}}\Delta y$ under the K extra equalities $y = U^{\mathsf{T}}w$: a QP with a diagonal Hessian and a sparse constraint matrix in $n + K$ variables. Each grid point is compared with the exact frontier at the same λ, in its weights and in what a user reads off it, its expected return and its variance.

In this form the warm-started **OSQP** grid costs 1.7× to 2.4× the factor trace up to $n = 160$ and 9× to 26× beyond, 18 s against 1.2 s at $n = 2560$. The ratio is not monotone in n because **OSQP**'s iteration count is not: from 51 iterations per grid point at $n = 20$ it rises to 1867 at $n = 640$, falls to 948 at $n = 1280$ and rises again to 2166 at $n = 2560$. Its weights are within 4×10^{-8} of the trace, and return and variance within a relative 3×10^{-8}. The interior-point solver **Clarabel** (Goulart and Chen 2024), which cannot warm-start but needs only 8 to 12

iterations per point at every size, is faster: its grid costs 1.5× to 3.9× the factor trace, 1.8 s at $n = 2560$ (1.8 to 3.0 s over the three runs). Its accuracy is that of its tolerance. At the default settings the weights are within 2×10^{-3} of the trace and the returns within a relative 10^{-5}, but the variances only within a relative 4×10^{-3}, so the risk read off a grid portfolio is off in the third digit; tightening its tolerances to 10^{-12} brings weights and variances to 2×10^{-5} and 4×10^{-5}, at 1.1× the time.

On real data the picture changes with the free set. On the rank-20 factor model of the S&P 500 (Section 7.7), whose free set stays below 56 of 494 assets, the factor trace takes 18 ms for 89 turning points, while the **Clarabel** grid of 100 points takes 0.42 s, 24× as long, with variances off by a relative 1.5×10^{-3}, and the **OSQP** grid 1.7 s, 95× as long, accurate to round-off (`make_figures.py osqp`).

Two qualifications apply. The grid points are independent, so a grid parallelizes perfectly in principle, while the trace is sequential. Split over a warm pool of 14 processes, one per core, the **Clarabel** grid runs 8.6× faster on the synthetic market at $n = 2560$, in 0.21 s, and so beats the factor trace (1.2 s) in wall-clock time; on the S&P 500 it runs 6.9× faster, in 58 ms, still three times the trace (18 ms; `make_figures.py parallel-grid`). And a grid returns 100 portfolios, the trace every turning point and hence, by interpolation, every efficient portfolio. With large free sets, then, an interior-point grid on the factor form costs about as much as the exact trace serially, and less in parallel; with the small free sets of the S&P 500 it costs an order of magnitude more. For a single risk target a general solver is the natural choice; for the whole frontier, **cvxcla** returns it complete and exact to round-off at no more than the cost of a grid.

7.6. A parametric QP solver

qpOASES (Ferreau *et al.* 2014) solves parametric QPs by an online active-set strategy: a hot start from one QP to the next follows the homotopy between them and changes the working set at every breakpoint on the way. Hot-started from the maximum-return end to $\lambda = 0$, it walks the same path as the CLA, so it is both the closest alternative and an independent check. We run it through **CasADi** (Andersson, Gillis, Horn, Rawlings, and Diehl 2019), solving once above the first finite turning point and hot-starting once to $\lambda = 0$ (`make_figures.py scaling`).

The two codes follow the same path. On every scaling problem **qpOASES** makes exactly as many working-set changes as **cvxcla** reports turning points $(21, 42, \ldots, 2561)$, and its endpoint matches the minimum-variance portfolio to $\sim 2 \times 10^{-16}$.

What differs is what each code accepts and returns. The constraint model of **qpOASES**, box bounds and two-sided general rows $\underline{a} \leq Aw \leq \bar{a}$, is at least as general as that of **cvxcla**, and its homotopy can move the bounds as well as the linear term. It takes the Hessian as a matrix, however, so a factor model reaches it only through auxiliary variables; a gross-exposure cap has to be rewritten by the user, with a singular Hessian in the lifted variables; and the turning points it passes are not returned through **CasADi**, only the endpoint. **cvxcla** returns every turning point, takes the covariance as an operator, and performs the lift itself.

For speed we compare like with like, both codes on the dense Σ; the time of **qpOASES** includes **CasADi**'s interface, which we did not separate. On small problems **qpOASES** is faster, by a factor of several up to $n = 80$, where **cvxcla**'s Python per-step overhead dominates (the event scan and the bookkeeping run in **NumPy**, with no compiled inner loop, which would remove

most of that overhead); the two meet near $n = 200$. Beyond that **qpOASES** takes about 3× as long as the dense backend at $n = 640$ (1.5 s) and 8× at $n = 2560$ (314 s; 8× to 12× over the three runs).

7.7. Empirical example: the S&P 500 frontier

The experiment runs off a frozen snapshot, kept in the package repository and in the replication materials (it is not part of the installed package): $N = 494$ assets over $T = 1213$ trading days, 2021-07-30 to 2026-05-29, stored as `sp500_pct_returns.parquet` (SHA-256 `b5faa522...9937e`) and produced once by `fetch_sp500.py`, retained for provenance only. That script takes the index constituents listed on the day it was run, downloads their split- and dividend-adjusted closes from **yfinance**, drops names missing more than 5% of the trading days, and forward-fills short gaps. The universe is the index at the end of the window, with survivorship bias, so we use it only as a realistic covariance; it says nothing about investment performance. From the full-history sample moments (condition number $\approx 1.1 \times 10^5$) the CLA traces the long-only frontier, 96 turning points, in about 25 ms. Its free set is small, at most 55 of the 494 assets and 38 at the median, so each dense step is cheap; a rank-20 factor approximation traces a near-identical frontier (89 points) in about 18 ms, against 25 ms for the dense backend on the same rank-20 matrix. The frontier spans annualized returns of 0.14–0.89 over volatilities of 0.10–0.87 (`make_figures.py real-frontier`). **cvxcla** controls only the computational error: an exact frontier of a noisy estimate is still noisy, and which estimate to trace is a statistical choice outside the package.

8. Conclusion

The Critical Line Algorithm computes the whole mean–variance frontier by walking its piecewise-linear path from the maximum-return to the minimum-variance portfolio. **cvxcla** implements that walk so that it never needs the covariance as a matrix: every step, including the start on a tied face, the gross-exposure lift and the singularity guard, reaches Σ only through the five members of the **cvx-linalg** operator protocol. Dense, factor, data-matrix and user-written backends therefore run the same loop and return the same turning points. The loop handles box bounds, general linear equalities and inequalities, and a gross-exposure cap that the package lifts itself; a least-index tie-break makes it terminate provably under standard assumptions; every turning point is checked for feasibility; and an independent KKT certificate, which trusts neither the code's partition nor its multipliers, passes on every trace it was run on, up to 2560 assets.

A structured backend pays where the free set is large. On a factor market of 2560 assets in which every asset is free at some point, the factor backend traces the frontier about 30 times faster than the dense one, in under 1 MiB of working memory, and an interior-point grid of 100 portfolios on the same factor form costs about as much, accurate only to its tolerance. On the S&P 500, where few assets are free at once, the gain over the dense backend is about 1.4×, and the same grid takes more than twenty times as long as the exact trace.

Three limitations remain. Below about 200 assets the per-step overhead of Python dominates, and **qpOASES**, which follows the same path, is faster. A change in μ or Σ needs a full new trace, with no warm start from the old one. And the KKT certificate belongs to the replication script, not yet to the package.

Computational details

All results in this paper were computed with **cvxcla** 2.7.1 (Schmelzer 2026d), its linear-algebra dependency **cvx-linalg** 1.1.2 (Schmelzer 2026c), and Python 3.14.6 on an Apple M4 Pro (14 cores, 48 GB of memory) running macOS 15.6.1. **NumPy** 2.4.6 and **SciPy** 1.17.1 were linked against Apple's Accelerate BLAS, and the first turning point under a general equality system is found with the **HiGHS** solver through `scipy.optimize.linprog`. The comparisons use **OSQP** 1.1.3, **Clarabel** 0.11.1, the **qpOASES** bundled with **CasADi** 3.8.1, and R 4.6.1 with the R package **CLA** 0.96-3. The figures use **matplotlib** 3.11.0, and the return data are read with **pandas** 3.0.3 and **pyarrow** 24.0.0.

Software availability. **cvxcla** is released under the MIT license. Release 2.7.1 is archived on Zenodo (Schmelzer 2026d) under `doi:10.5281/zenodo.22209208`; the source is developed at `https://github.com/cvxgrp/cvxcla`, and every source link in this paper points to the release tag `v2.7.1`. Its one non-standard dependency, **cvx-linalg** (Schmelzer 2026c), is also MIT-licensed and on PyPI, developed at `https://github.com/Jebel-Quant/linalg` with its own test suite and documentation; it has no archived release of its own, and the replication script pins version 1.1.2, whose source links in Section 5 point to the tag `v1.1.2`. The package declares only lower bounds on its dependencies, and its continuous integration also tests them at those bounds; the experiment script pins the exact versions above. The test suite, some 600 unit and property-based tests (the latter with **Hypothesis**), enforces full line coverage of the library.

Reproducing the results. Every figure, table, and number is produced by one script, `make_figures.py`, which the replication materials of this article hold together with the returns snapshot. It is the experiment script of release 2.7.1 (`make_figures.py` in the **cvxcla** repository), less the timing of the **PyPortfolioOpt** CLA, which this paper does not report and the separate note (Schmelzer 2026e) takes up, and less two steps whose results this paper does not use (covariance estimators and resampled frontiers), and extended for this article: by the worked example of Section 6.4, the factor-form grids of Section 7.5, the comparison with the R package **CLA**, the drift of the incremental backend, the single-thread timings, the parallel grid, the Gram backend's certificate and timing, the KKT certificate of the largest traces, the wrong-start test of Appendix B, and the tolerance study of Appendix A.1; the dense **OSQP** grid of the release, which this paper does not use, is removed. Every step named in the text is run by this script against the released package, unchanged; the links on step names point to the release's script, which lacks these extensions. The script declares the versions above as inline (PEP 723) metadata, so with **uv** installed

```
uv run make_figures.py all
```

installs them into an isolated environment and rebuilds everything. A target name in place of `all` rebuilds one artifact, and `all --quick` skips the two scaling sweeps (Sections 7.2 and 7.4) and the single-thread timings, which take most of the run, under an hour on the machine above. Random inputs are drawn from fixed seeds and the S&P 500 returns are a committed snapshot (Section 7.7), so the run needs no network access once the packages are installed. Timings and memory were measured on this machine only and will differ elsewhere; the turning-point counts and accuracy figures should agree up to round-off. On Linux (Debian,

Python 3.12, **NumPy**'s bundled OpenBLAS, in a container on the same machine) the six correctness checks of Section 7.1 and Appendix A find the same turning points; the worst KKT residual is 3.5×10^{-13} against 1.1×10^{-13}, and only round-off-level counts move, such as which zero-length segments of the 130/30 trace occur and how often the projection of Appendix B fires. In the same container the step `blas` times the $n = 2560$ traces of Section 7.2 (dense 234 s, factor 1.5 s); these are the only timings not taken on macOS.

Acknowledgments

The author thanks Stephen Boyd, Trevor Hastie and Philipp Schiele. The first versions of the solver were implemented while the author stayed for a sabbatical at Boyd's group at Stanford University.

A. Implementation details

A.1. The event guards

Floors and window. Slopes are tested against a floor of $\sqrt{\varepsilon_{\text{mach}}}$ in the problem's own units, not against the user tolerance, which would let a tiny but genuine slope go unnoticed. With $m = \|\mu\|_\infty$ and $\sigma = \|\Sigma \mathbf{1}\|_\infty / n$, λ has the scale σ/m. Weight slopes β_i and slack slopes $g_j^{\mathsf{T}} \beta$ carry units of $1/\lambda$ and are tested against $\sqrt{\varepsilon_{\text{mach}}}\, m/\sigma$; multiplier slopes δ_i, η_j^{β} carry the units of μ and are tested against $\sqrt{\varepsilon_{\text{mach}}}\, m$. The λ window is a relative 10^{-10} taken against $\max(|\lambda|, \sigma/m)$, so that it does not vanish as $\lambda \to 0$.

Units and the floor. The trace does not depend on the units. Rescaling the returns by c and the covariance by s, each from 10^{-6} to 10^{6} (`make_figures.py validate-scaling`), leaves all 29 turning points of the 40-asset problem in place: the weights agree to 8×10^{-15}, and $\lambda c/s$ to a relative 1.2×10^{-13}. Multiplying the slope floors by any factor from 10^{-6} to 10^{4} leaves the traces of the 40- and 494-asset S&P 500 problems, the 600-day window and the 320-asset factor market bit for bit unchanged. Only at 10^{6} does the floor start to swallow genuine slopes: the 40-asset trace is still unchanged, the 600-day window's weights move by 7×10^{-2}, and the trace of the 494-asset problem and of the factor market is declined. So on these inputs genuine slopes and round-off are about ten orders of magnitude apart.

The user tolerance. `tol` enters the trace in three places: it decides which weights sit on a bound at the linear-programming first vertex and at the start on a face of tied maximum returns (Section 3), and it bounds the violation of a bound, a row of G or the gross cap that validation accepts after the projection. The equality rows are validated against a fixed 10^{-7}, independent of `tol`. Along the walk the events place blocked weights exactly on their bounds, so it plays no part there. From 10^{-3} to 10^{-10} it leaves bit for bit unchanged the four traces above, the three constrained frontiers of Section 7.1 (whose first vertex is a linear program for the neutrality row, and whose 130/30 book is validated through the lift), and three tied-means problems with 20, 60 and 120 assets, drawn from one seed and not members of the

family of Appendix B, whose 60-asset instance is a different problem. The 120-asset one is the exception: at 10^{-3} its start on the tied face is declined with a `DegenerateProblemError`, and from 10^{-4} down it is unchanged. A loose `tol` therefore weakens validation and can misread the start of a tied problem; on these problems it never changed a trace that completed. A turning point lost to the floor would leave the later ones feasible but not optimal, which the KKT certificate reports as a stationarity or dual-feasibility residual.

A.2. The gross-exposure lift

Both legs of an asset are never free together on the path. Write g_i for the i-th entry of $\Sigma w - \lambda\mu + A^\mathsf{T}\nu + G^\mathsf{T}\eta_G$ and $\eta \geq 0$ for the multiplier of the cap; the reduced gradients of the two legs are $g_i + \eta$ and $-g_i + \eta$. A free long leg has $g_i = -\eta$, so its short leg's reduced gradient is 2η; a long leg at its upper bound has $g_i + \eta \leq 0$, so its short leg's is at least 2η (and symmetrically). The idle leg can therefore enter only where η vanishes, at the release of the cap, and the event search drops that competing leg event so the release fires.

At the first vertex, overlapping legs (the cap tight with a zero multiplier) are removed by subtracting $\min(x_i^+, x_i^-)$ from both, which leaves w and every other constraint unchanged; a cap at the smallest feasible gross exposure ($c = 1$ under a full budget), which fails Slater's condition, is handled by solving $\min \mathbf{1}^\mathsf{T}x$ first and pinning every leg with a positive reduced cost.

A.3. The degenerate first vertex

The maximum-return vertex can be degenerate: every weight on a bound (any inequality row on a long-only problem whose best asset takes the whole budget), or a duplicated tight row. Its partition is then read off the linear program's duals, generalizing the budget rule of Section 3. At a degenerate vertex these are not unique, and the solution the simplex reports can show too few zero reduced costs to free enough assets. For example, with three assets, $\mu = (3, 2, 1)$, long-only, the budget and the row $w_2 = 0$, **HiGHS** reports duals that free only the first asset at $w = (1, 0, 0)$, whereas the equally optimal multipliers 1 on both rows free the other two. The code therefore first moves the reported duals to a vertex of the optimal dual set, stepping along directions that keep the tight dual constraints tight until they determine the multipliers. A basis is read off that dual vertex: the assets strictly inside their bounds, then the tight rows with zero multiplier, which are released, then the assets on a bound with zero reduced cost, in order of decreasing return, which are freed. The active rows then have a square, nonsingular free block, the first segment starts at the vertex itself, and the implied multipliers are those of the dual vertex, so the partition is dual feasible. The optimal dual set has no vertex, and the vertex is refused, when a tight row touches only assets fixed at both bounds.

Nearly dependent equality rows, such as $\mathbf{1}^\mathsf{T}w = 1$ and $(\mathbf{1} + \epsilon v)^\mathsf{T}w = 1$, which encode $v^\mathsf{T}w = 0$ only at scale ϵ, would be enforced by the linear program only to its tolerance divided by ϵ. When the row-normalized equality matrix has a reciprocal condition number below 10^{-6} they are handed to **HiGHS** in orthonormal rows, $Q^\mathsf{T}w = R^{-\mathsf{T}}b$ with $A^\mathsf{T} = QR$.

B. Numerical robustness: degeneracy and ties

Round-off. A candidate turning point can be optimal to solver precision but not exactly feasible: round-off accumulated over hundreds of turning points can leave a free weight a hair outside its box. `CLA._emit` projects such a point back onto the feasible set before it validates it. The projection is a numerical repair. It keeps the point optimal to working accuracy only when the correction is small and lies in near-flat directions of the objective, where round-off usually sits, so we measure it below.

For the plain budget with no active inequality row the projection onto the capped simplex is a single shift of the weights, found by bisection, followed by a clip. Under a general A or an active row of G the code alternates a box clip with the affine projection onto $\{w : Cw = c\}$, $C = [\,A; G_{\mathcal{S}}\,]$, written in orthonormal rows so that its Gram matrix is the identity rather than CC^{T}. It usually converges in one iteration. It stops once the box is violated by at most 10^{-12}, and raises a `ProjectionError` if 100 iterations do not get there.

The measurements of `make_figures.py validate-projection` support this for the problems tested. On the rank-deficiency sweep of Figure 6 and on sample covariances of the S&P 500 (Section 7.7) over the last $T = 120$ to 1213 days, the projection changes between none and four turning points per trace; the largest weight correction is 9.3×10^{-15}, the largest relative change of the objective 8.7×10^{-19}, and every projected point satisfies the box and the budget to 2.2×10^{-16}. The KKT certificate (Table 5) bounds every residual on the segments between projected points by 2×10^{-13}, and every completed trace of the sweep matches a reference QP in objective to $\sim 10^{-13}$.

Rank deficiency. The two regimes are told apart by the *conditioning* of the free block. While it stays numerically full rank any infeasibility is round-off and is projected away. Once the free set outgrows the covariance rank, $\Sigma_{\mathcal{F}\mathcal{F}}$ is singular, but the reduced system (2) need not be: the active rows $C_{\mathcal{F}}$ hold the free weights on their null space, and the system is uniquely solvable when $Z^{\mathsf{T}}\Sigma_{\mathcal{F}\mathcal{F}}Z$ is positive definite, where the columns of Z span $\{v : C_{\mathcal{F}}v = 0\}$. A guard reads the reciprocal condition number of $\Sigma_{\mathcal{F}\mathcal{F}}$ from its eigenvalues, a quantity that, unlike the size of a violation, is stable across BLAS builds. When it falls below 10^{-12} the guard reads that of $Z^{\mathsf{T}}\Sigma_{\mathcal{F}\mathcal{F}}Z$ instead, the segment is solved on the null space, and the trace is declined with a diagnosis only when that block is singular too, since projecting its output could silently yield a suboptimal frontier. A guard on $\Sigma_{\mathcal{F}\mathcal{F}}$ alone would refuse the frontier of a sample covariance from fewer observations than assets as soon as its free set outgrows the data rank (`make_figures.py degeneracy`). Singular constraint rows, at the first vertex or later in the trace, are declined in the same way (Section 3, Section 4.3).

Conditioning. To see the guard on its own we prescribe the spectrum, $\Sigma = Q\,\mathrm{diag}(\lambda)Q^{\top}$ with 60 eigenvalues spread geometrically from 1 to $1/\kappa$, under a long-only budget, and certify each trace (`make_figures.py validate-conditioning`). With a random orthogonal Q the free blocks see only part of the spectrum, their worst condition number stays at 5.9×10^{9} for $\kappa = 10^{16}$, and every trace completes with KKT residuals below 2.5×10^{-14}. With $Q = I$ the free block inherits the full spread: traces up to $\kappa = 10^{12}$ complete and are certified (residuals up to 5.5×10^{-12}), and from 3×10^{12} to 10^{14} every trace is declined at the guard, so here its value decides the outcome. At $\kappa = 10^{16}$ the smallest eigenvalues are zero to working precision; the trace completes again, in 10 turning points, and is certified (1.1×10^{-15}). It and **OSQP** reach different weights, up to 0.94 apart, yet both meet the optimality conditions, so there the optimum is not unique.

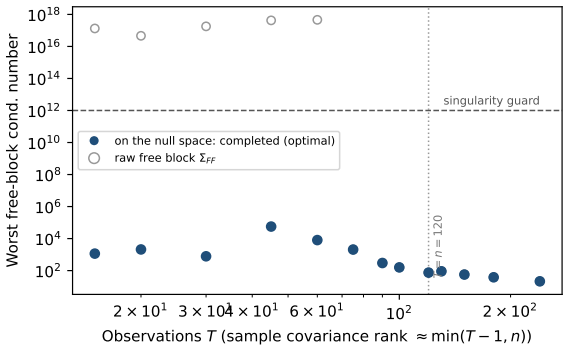


Figure 6: A rank-deficiency sweep at $n = 120$, the long-only covariance estimated from T observations. Grey rings are the worst condition number of the raw free block $\Sigma_{\mathcal{F}\mathcal{F}}$ over a trace; blue circles that of $Z^{\mathsf{T}}\Sigma_{\mathcal{F}\mathcal{F}}Z$, the block on the budget's null space, which is what the guard (dashed, 10^{12}) reads. Below $T = 75$ the raw block is singular ($\sim 10^{17}$) while the reduced block stays below 6×10^4, so every trace completes, and every completed frontier matches a reference QP to 3×10^{-13}. A guard on the raw block would decline the five windows with $T \leq 60$.

The reference QP is the less accurate of the two: OSQP leaves KKT residuals near 10^{-9} and differs from the trace by up to 0.2 in a weight in the diagonal family, where the KKT conditions only weakly determine the weights along directions of tiny variance. None of the 192 reference solves failed.

Event ties. Ties are handled by the least-index rule of Section 4.5. Across four deliberately degenerate families of 32 instances each (n up to 240; `make_figures.py tie-degeneracy`), a trace counts only if it is optimal: every turning point must pass the KKT certificate of Table 5 to 10^{-8}, so a frontier that ends but is wrong is reported as such. Exactly tied means give 31 certified traces; the other instance ($n = 60$) is declined partway along the frontier, at a degenerate turning point whose multipliers are not unique. Exchangeable pairs (identical means and covariance rows) give 32 certified traces, and the tie-break never frees both members of a pair. A family of $p \approx n/3$ group caps gives 32, with up to 53 rows active at once, and an over-constrained family of $p = n$ overlapping caps, whose maximum-return vertex is degenerate, gives 32, with up to 240 rows active at once. No trace is off the frontier.

Counting completed traces would not be enough in principle: the loop checks each turning point for feasibility, not for optimality, so only the certificate can tell a wrong trace from a correct one. A wrong start is the natural test. Started at the vertex the solver returns instead of the point of the tied face that Section 3 computes (with `CLA._face_minimiser` bypassed), none of the 64 tied-means and exchangeable traces completes: all are declined with an error, none returns a frontier off the efficient one.

The tied-means and group-cap families, the full-rank windows of Figure 6 and both conditioning families satisfy the hypotheses of Proposition 1 at every turning point, so for them termination is the theorem. The others do not. In the exchangeable family all 1751 tied turning points satisfy the hypotheses. The overlapping-cap traces pass 2637 ties, all with dependent constraints, so for them only the weaker guarantees of Section 4.5 hold. The nine

rank-deficient windows of Figure 6 have a singular Σ, outside $\Sigma \succ 0$; all nine complete, the five with $T \leq 60$ by solving on the budget's null space. No trace used more than 1.5% of the iteration cap $100(n+p+1)$.

C. Proof of Proposition 1

Leaving is permanent. A partition (free set, blocked bounds, active rows $\mathcal{S}$) is valid at λ when its segment respects the bounds and the inactive rows and its multipliers have the right signs. Each of these conditions is affine in λ. An event fires at $\lambda^\star$ only when one of them is zero there and its slope is nonzero with the sign that violates it below $\lambda^\star$ (the strict inequalities of Table 6); a condition with zero slope is constant and never fires. An affine function with nonzero slope that crosses zero stays on the wrong side, so a partition left at $\lambda^\star$ is invalid at every $\lambda < \lambda^\star$. In floating point the slope floor of Appendix A.1 stands in for the test that a slope is nonzero. Since λ never increases, a partition can recur only at the same $\lambda^\star$, within a tie.

The direction problem at a tie. Fix $\lambda^\star$. Because $\Sigma \succ 0$ the optimiser $w^\star = w(\lambda^\star)$ is unique, and by linear independence so are its multipliers. Constraints with slack, or holding with a nonzero multiplier, cannot fire at $\lambda^\star$, and every partition the walk visits there keeps their status; let E stack the gradients of the latter together with A. The partitions differ only on the set T of tied constraints, those holding with a zero multiplier, written $c_t^\mathsf{T} w \leq \text{const}$ (with $c_t = \pm e_i$ for a bound). Below $\lambda^\star$ the walk follows a segment $w(\lambda^\star - \tau) = w^\star + \tau d$, $d = -\beta$, on which the multipliers are affine in τ too, $\nu(\tau) = \nu^\star + \tau\nu'$. Substitute both into the KKT conditions of (1) and keep the terms of order τ. Stationarity gives $\Sigma d + \mu + E^\mathsf{T}\nu'_E + C_T^\mathsf{T}\nu'_T = 0$. A constraint in E keeps a nonzero multiplier for small τ, so it stays active and $Ed = 0$. A tied constraint has slack $-\tau\, c_t^\mathsf{T} d$ and multiplier $\tau\,\nu'_t$, so feasibility, the sign of the multiplier, and complementary slackness require $c_t^\mathsf{T} d \leq 0$, $\nu'_t \geq 0$, and $(c_t^\mathsf{T} d)\,\nu'_t = 0$. These are the KKT conditions of the strictly convex QP

$$\min_d \ \tfrac{1}{2}\, d^\mathsf{T}\Sigma d + \mu^\mathsf{T} d \quad \text{subject to} \quad Ed = 0, \quad c_t^\mathsf{T} d \leq 0 \ \ (t \in T),$$

so d is its unique solution, the classical directional derivative of a parametric program without strict complementarity (Jittorntrum 1984; Bonnans and Shapiro 2000).

The complementarity problem. Let E have linearly independent rows, let Z be a basis of its null space, and let C_T collect the columns c_t, $t \in T$. Every d with $Ed = 0$ is $d = Zy$, and the direction problem becomes $\min_y \tfrac{1}{2}\, y^\mathsf{T} H y + \mu^\mathsf{T} Z y$ subject to $C_T^\mathsf{T} Z y \leq 0$, with $H = Z^\mathsf{T}\Sigma Z \succ 0$ because $\Sigma \succ 0$ and Z has full column rank. Its KKT conditions, with slacks $s = -C_T^\mathsf{T} Z y$ and multipliers $\nu \geq 0$, eliminate $y = -H^{-1}Z^\mathsf{T}(\mu + C_T\nu)$ and leave the linear complementarity problem

$$s = q + M\nu, \qquad s \geq 0,\ \nu \geq 0,\ s^\mathsf{T}\nu = 0, \qquad M = C_T^\mathsf{T} Z H^{-1} Z^\mathsf{T} C_T, \quad q = C_T^\mathsf{T} Z H^{-1} Z^\mathsf{T}\mu.$$

M is positive semidefinite, and positive definite exactly when $Z^\mathsf{T} C_T$ has full column rank. Suppose $Z^\mathsf{T} C_T\, x = 0$. Then $C_T x$ lies in the null space of Z^T, which is the row space of E, so

Event (column)	Status of t	Basic variable	Fires at $\lambda^\star$ when
free weight reaches its upper bound (0)	released	s_t	$\beta_i < 0$
free weight reaches its lower bound (1)	released	s_t	$\beta_i > 0$
blocked weight leaves its upper bound (2)	enforced	ν_t	$\delta_i < 0$
blocked weight leaves its lower bound (3)	enforced	ν_t	$\delta_i > 0$
inactive row binds (0, row block)	released	s_t	$g_j^\mathsf{T}\beta < 0$
active row releases (1, row block)	enforced	ν_t	$\eta_j^\beta > 0$

Table 6: The six event types of the event scan and the complementary pair each one pivots. A coordinate's status admits one event type at a time, so a coordinate index identifies the pair.

$C_T x = E^\mathsf{T} z$ for some z, and $[\,E^\mathsf{T},\ C_T\,](-z, x) = 0$. The hypothesis that the gradients of E and of the tied constraints are linearly independent forces $z = 0$ and $x = 0$. Hence $M \succ 0$, and in particular M is a P-matrix: every principal minor is positive.

The correspondence with the implementation. Each tied constraint t contributes one complementary pair (s_t, ν_t), and a partition visited at $\lambda^\star$ makes exactly one of the two basic: s_t when the constraint is released (a free weight, an inactive row), ν_t when it is enforced (a blocked weight, an active row). The β-system of that partition (Section 4.1) is the equality-constrained form of the direction problem with the enforced constraints held, so its solution is the basic solution of the complementarity problem. Table 6 lists the six event types of the code (`_events.py`). Each fires at $\lambda^\star$ exactly when the basic variable of its pair is negative: a released constraint whose slack would turn negative as λ falls, or an enforced one whose multiplier would.

The pivoting rule. When several events fire at $\lambda^\star$, `select_next_event` (`pathtracer.py`) flips the one with the lowest coordinate index, assets before inequality rows, and changes the status of that coordinate alone. By the correspondence above this is a simple principal pivot on the lowest-index pair whose basic variable is negative, in a fixed ordering of the pairs: Murty's least-index method (Murty 1974), which terminates on every linear complementarity problem with a P-matrix, for every q (Murty 1974; Cottle, Pang, and Stone 1992). The pivot depends only on the current partition, so a partition repeated at $\lambda^\star$ would repeat forever; none is, and the walk leaves $\lambda^\star$ after finitely many steps with a feasible basic solution, along a segment of positive length.

Termination. Each segment of positive length uses a partition never used before, and there are finitely many partitions; each tie between two such segments takes finitely many steps.

References

Andersson JAE, Gillis J, Horn G, Rawlings JB, Diehl M (2019). "**CasADi**: A Software Framework for Nonlinear Optimization and Optimal Control." *Mathematical Programming Computation*, **11**(1), 1–36. doi:10.1007/s12532-018-0139-4.

Best MJ (1996). "An Algorithm for the Solution of the Parametric Quadratic Programming Problem." In H Fischer, B Riedmüller, S Schäffler (eds.), *Applied Mathematics and Parallel Computing*, pp. 57–76. Physica-Verlag, Heidelberg. doi:10.1007/978-3-642-99789-1_5.

Best MJ (2010). *Portfolio Optimization*. Chapman & Hall/CRC, Boca Raton, FL.

Bonnans JF, Shapiro A (2000). *Perturbation Analysis of Optimization Problems*. Springer-Verlag, New York. doi:10.1007/978-1-4612-1394-9.

Cajas D (2021). "**Riskfolio-Lib**: Portfolio Optimization and Quantitative Strategic Asset Allocation in Python." Python package version 7.4.0. URL https://github.com/dcajasn/Riskfolio-Lib.

Cottle RW, Pang JS, Stone RE (1992). *The Linear Complementarity Problem*. Academic Press, Boston, MA.

Ferreau HJ, Kirches C, Potschka A, Bock HG, Diehl M (2014). "**qpOASES**: A Parametric Active-Set Algorithm for Quadratic Programming." *Mathematical Programming Computation*, **6**(4), 327–363. doi:10.1007/s12532-014-0071-1.

Fu A, Narasimhan B, Boyd S (2020). "**CVXR**: An R Package for Disciplined Convex Optimization." *Journal of Statistical Software*, **94**(14), 1–34. doi:10.18637/jss.v094.i14.

Goulart PJ, Chen Y (2024). "**Clarabel**: An Interior-Point Solver for Conic Programs with Quadratic Objectives." arXiv:2405.12762. doi:10.48550/arXiv.2405.12762.

Hirschberger M, Qi Y, Steuer RE (2010). "Large-Scale MV Efficient Frontier Computation via a Procedure of Parametric Quadratic Programming." *European Journal of Operational Research*, **204**(3), 581–588. doi:10.1016/j.ejor.2009.11.016.

Jittorntrum K (1984). "Solution Point Differentiability without Strict Complementarity in Nonlinear Programming." *Mathematical Programming Study*, **21**, 127–138. doi:10.1007/BFb0121215.

Markowitz H, Starer D, Fram H, Gerber S (2021). "Avoiding the Downside: A Practical Review of the Critical Line Algorithm for Mean-Semivariance Portfolio Optimization." In *Handbook of Applied Investment Research*, chapter 17. World Scientific. doi:10.1142/9789811222634_0017.

Markowitz HM (1952). "Portfolio Selection." *The Journal of Finance*, **7**(1), 77–91. doi:10.2307/2975974.

Markowitz HM (1956). "The Optimization of a Quadratic Function Subject to Linear Constraints." *Naval Research Logistics Quarterly*, **3**(1–2), 111–133. doi:10.1002/nav.3800030110.

Markowitz HM (1959). *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, New York.

Markowitz HM, Todd GP (2000). *Mean-Variance Analysis in Portfolio Choice and Capital Markets*. Frank J. Fabozzi Associates, New Hope, PA.

Martin RA (2021). "**PyPortfolioOpt**: Portfolio Optimization in Python." *Journal of Open Source Software*, **6**(61), 3066. doi:10.21105/joss.03066.

Murty KG (1974). "Note on a Bard-Type Scheme for Solving the Complementarity Problem." *Opsearch*, **11**(2–3), 123–130.

Nicolini C, Manzi M, Delatte H (2025). "**skfolio**: Portfolio Optimization in Python." arXiv:2507.04176; Python package version 1.12.5. doi:10.48550/arXiv.2507.04176.

Niedermayer A, Niedermayer D (2010). "Applying Markowitz's Critical Line Algorithm." In *Handbook of Portfolio Construction*, pp. 383–400. Springer-Verlag, Boston, MA. doi:10.1007/978-0-387-77439-8_12.

Perold AF (1984). "Large-Scale Portfolio Optimization." *Management Science*, **30**(10), 1143–1160. doi:10.1287/mnsc.30.10.1143.

Schmelzer T (2026a). "The Constrained LASSO Is a Portfolio: A Numerical Check of the CLA–LASSO Identity under Constraints." *Journal of Reproducible Statistics*, **1**, 10. doi:10.68365/jrepstat.2026.10.

Schmelzer T (2026b). "The Efficient Frontier from a LASSO Solver: A Numerical Check of the CLA–LASSO Identity." *Journal of Reproducible Statistics*, **1**, 7. doi:10.68365/jrepstat.2026.7.

Schmelzer T (2026c). ***cvx-linalg**: Linear Algebra Utilities for Portfolio Optimization*. Python package version 1.1.2, URL https://github.com/Jebel-Quant/linalg.

Schmelzer T (2026d). ***cvxcla**: The Critical Line Algorithm with a Quadratic-Form Operator Interface*. doi:10.5281/zenodo.22209208. Python package version 2.7.1, URL https://github.com/cvxgrp/cvxcla.

Schmelzer T (2026e). "Revisiting the Open-Source Critical Line Algorithm of Bailey and López de Prado: Speed, Correctness and Robustness." Manuscript. URL https://github.com/cvxgrp/cvxcla/blob/paper/comparison.pdf.

Schmelzer T, Hastie T (2026). "The Critical Line Algorithm and the Constrained LASSO: One Curve, Two Literatures." arXiv:2609.25704. doi:10.48550/arXiv.2609.25704.

Sharpe WF (1963). "A Simplified Model for Portfolio Analysis." *Management Science*, **9**(2), 277–293. doi:10.1287/mnsc.9.2.277.

Shi Y, Maechler M (2024). ***CLA**: Critical Line Algorithm in Pure R*. R package version 0.96-3, URL https://CRAN.R-project.org/package=CLA.

Stein M, Branke J, Schmeck H (2008). "Efficient Implementation of an Active Set Algorithm for Large-Scale Portfolio Selection." *Computers & Operations Research*, **35**(12), 3945–3961. doi:10.1016/j.cor.2007.05.004.

Stellato B, Banjac G, Goulart P, Bemporad A, Boyd S (2020). "**OSQP**: An Operator Splitting Solver for Quadratic Programs." *Mathematical Programming Computation*, **12**(4), 637–672. doi:10.1007/s12532-020-00179-2.

Wolfe P (1959). "The Simplex Method for Quadratic Programming." *Econometrica*, **27**(3), 382–398. doi:10.2307/1909468.

Würtz D, Chalabi Y, Chen W, Ellis A (2009). *Portfolio Optimization with R/**Rmetrics***. Rmetrics Association and Finance Online, Zurich.

Affiliation:

Thomas Schmelzer
Jebel Quant Research
Abu Dhabi, United Arab Emirates
E-mail: thomas@jqr.ae
URL: https://github.com/cvxgrp/cvxcla
Preprint: doi:10.2139/ssrn.7583179

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd